第4章 串

提纲

CONTENTS

4.1 串的定义

4.2 串的存储结构

4.3 STL中的string

4.4 串的模式匹配

4.1 串的定义

- 串是由零个或多个字符组成的有限序列。记作str="$a_0a_1\cdots a_{n-1}$"（$n\geqslant 0$）。 串 ⊂ 线性表
- 串中所包含的字符个数n称为串长度，当$n=0$时，称为空串。
- 一个串中任意连续的字符组成的子序列称为该串的子串。
- 包含子串的串相应地称为主串。
- 若两个串的长度相等且对应字符都相等，则称两个串相等。

如：

“abcd” ≠ “abc”

“abcd” ≠ “abcde”

所有空串是相等的。

【例4.1】设s是一个长度为n的串，其中的字符各不相同，则s中的所有子串个数是多少？



- 空串是其子串，计1个。
- 每个字符构成的串是其子串，计n个。
- 每2个连续的字符构成的串是其子串，计n-1个。
- 每3个连续的字符构成的串是其子串，计n-2个。
- …
- 每n-1个连续的字符构成的串是其子串，计2个。
- s是其自身的子串，计1个。

子串个数
=1+n+(n-1)+…+2+1
=n(n+1)/2+1

例如，s="software"的子串个数=(8×9)/2+1=37。

4.1.2 串的抽象数据类型

ADT String
{
数据对象：
D={a_i | 0≤i≤n-1，n≥0，a_i为字符类型}
数据关系：
R={r}
r={<a_i，a_{i+1}> | a_i，a_{i+1}∈D， i=0，…，n-2}
基本运算：
StrAssign(cstr)：由字符串常量cstr创建一个串，即生成其值等于cstr的串。
StrCopy()：串复制，返回由当前串复制产生一个串。
getlength()：求串长，返回当前串中字符个数。
geti(i)：返回序号i的字符。
seti(i，x)：设置序号i的字符为x。
Concat(t)：串连接，返回一个当前串和串t连接后的结果。
SubStr(i，j)：求子串，返回串中从第i个字符开始的j个连续字符组成的子串。
InsStr(i，t)：串插入，返回串t插入到当前串的第i个位置后的子串。
DelStr(i，j)：串删除，返回当前串中删去从第i个字符开始的j个字符后的结果。
RepStr(i，j，t)：串替换，返回用串t替换当前串中第i个字符开始的j个字符后的结果。
DispStr()：输出字符串。
}

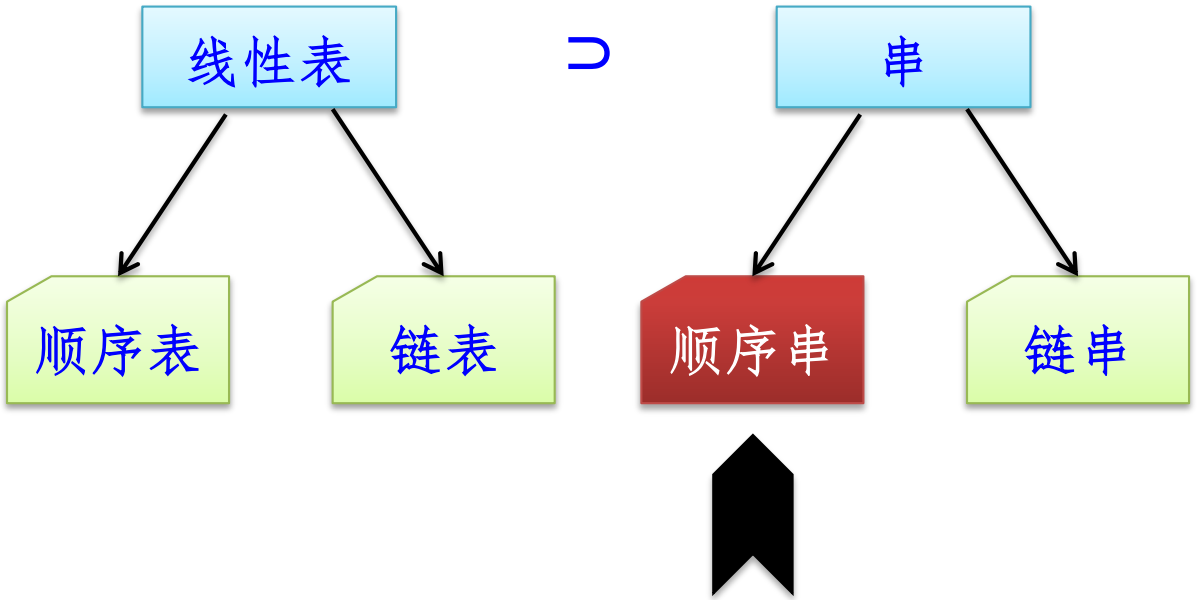
4.2 串的存储结构

串的实现方式

逻辑结构

↓ 映射

存储结构



4.2.1 串的顺序存储结构–顺序串

- 和顺序表一样，用一个data数组和一个整型变量length来表示一个顺序串，length表示data数组中实际字符的个数。
- 为了简单，data数组采用固定容量为MaxSize（可以模仿顺序表改为动态容量方式）。

顺序串类SqString

```
const int MaxSize=100;          //字符串的最大长度
class SqString                  //顺序串类
{
public:                         //为了简单将成员均设置为公有的
   char* data;                  //存放串中元素
   int length;                  //串长度
  //串的基本运算算法
};
```

【例4-1：】设计顺序串上实现串比较运算Strcmp(s，t)的算法。例如：

"ab" < "abcd"　　"abcd" < "abd"

解：算法思路如下：

（1）比较s和t两个串共同长度范围内的对应字符：

① 若s的字符＞t的字符，返回1；

② 若s的字符＜t的字符，返回-1；

③ 若s的字符=t的字符，按上述规则继续比较。

（2）当（1）中对应字符均相同时，比较s和t的长度：

① 两者相等时，返回0；

② s的长度＞t的长度，返回1；

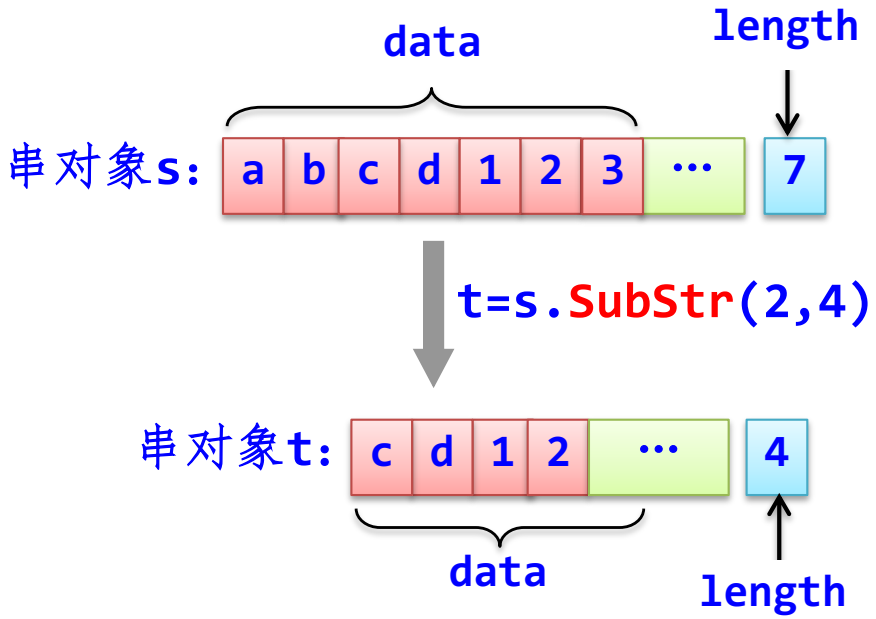
③ s的长度＜t的长度，返回-1。

```
int  Strcmp(SqString s, SqString t)
{  int i, comlen;
   if (s.length<t.length)  comlen=s.length; //求s和t的共同长度
   else comlen=t.length;
   for (i=0;i<comlen;i++)                   //在共同长度内逐个字符比较
      if (s.data[i]>t.data[i])
         return 1;
      else if (s.data[i]<t.data[i])
         return -1;
   if (s.length==t.length)                  //s==t
      return 0;
   else if (s.length>t.length)              //s>t
      return 1;
   else  return -1;                         //s<t
}
```

所有共同长度内的字符相同，哪个长哪个大

顺序串上的基本运算算法设计与顺序表类似，仅以求子串为例说明。

求子串：对于一个顺序串求序号*i*开始长度为*j*的子串。





实现：

- 先创建一个空串s。
- 当参数正确时，s子串的字符序列为data[i..i+j-1]，共j个字符，当i和i+j-1不在有效序序号0～length-1范围内时，则参数错误，此时返回空串。

```
SqString& SubStr(int i,int j)          //求子串运算算法
{  static SqString s;                  //新建一个空串
   if (i<0 || i>=length || j<0 || i+j>length)
      return s;                        //参数不正确时返回空串
   for (int k=i;k<i+j;k++)             //将str.data[i..i+j-1]->s
      s.data[k-i]=data[k];
   s.length=j;
   return s;                           //返回新建的顺序串
}
```

【例4.2】设计一个算法Strcmp(s，t)，以字典顺序比较两个英文字母串s和t的大小，假设两个串均以顺序串存储。



算法思路如下：

（1）比较s和t两个串共同长度范围内的对应字符：

① 若s的字符＞t的字符，返回1；

② 若s的字符＜t的字符，返回-1；

③ 若s的字符=t的字符，按上述规则继续比较。

（2）当（1）中对应字符均相同时，比较s和t的长度：

① 两者相等时，返回0；

② s的长度＞t的长度，返回1；

③ s的长度＜t的长度，返回-1。

【例4.2】设计一个算法Strcmp(s，t)，以字典顺序比较两个英文字母串s和t的大小，假设两个串均以顺序串存储。

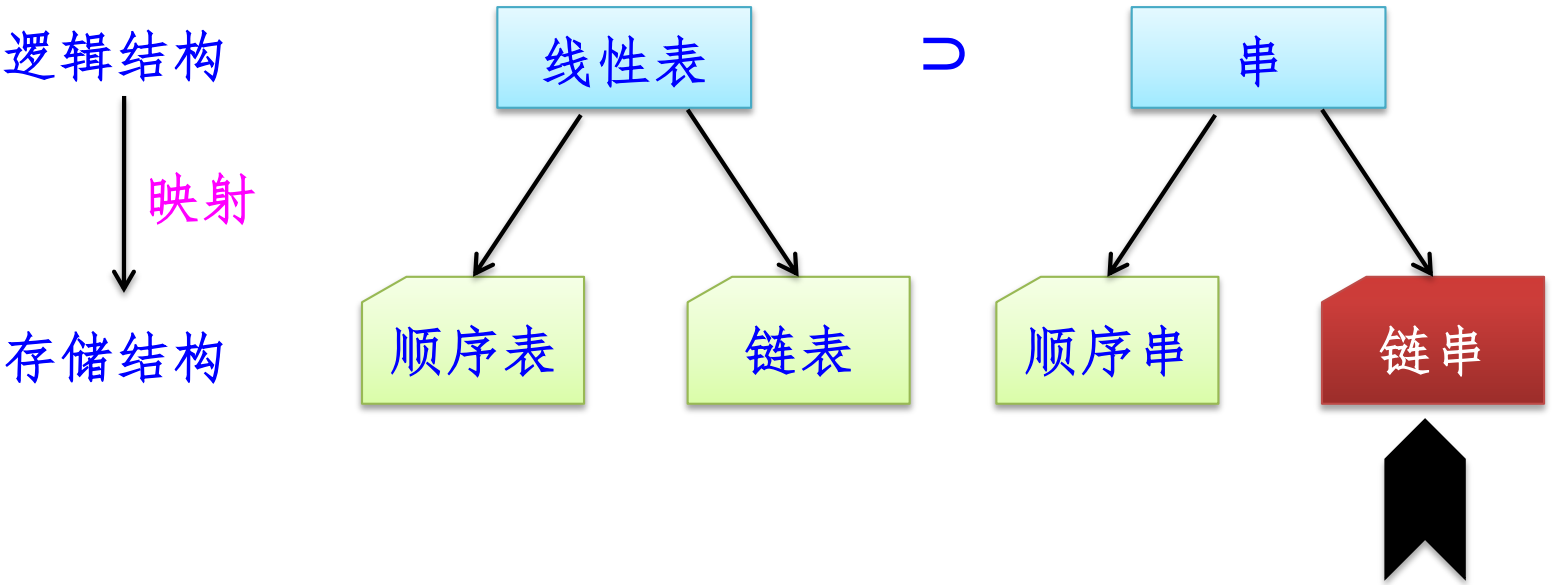



```
int Strcmp(SqString& s,SqString& t)              //比较串s和t的算法
{  int minl=min(s.getlength(),t.getlength()); //求s和t中最小长度
   for (int i=0;i<minl;i++)                      //共同长度内逐个字符比较
      if (s.data[i]>t.data[i]) return 1;
      else if (s.data[i]<t.data[i]) return -1;
   if (s.getlength()==t.getlength())             //s==t
      return 0;
   else if (s.getlength()>t.getlength())         //s>t
      return 1;
   else                                          //s<t
      return -1;
}
```

所有共同长度内的字符相同，哪个长哪个大

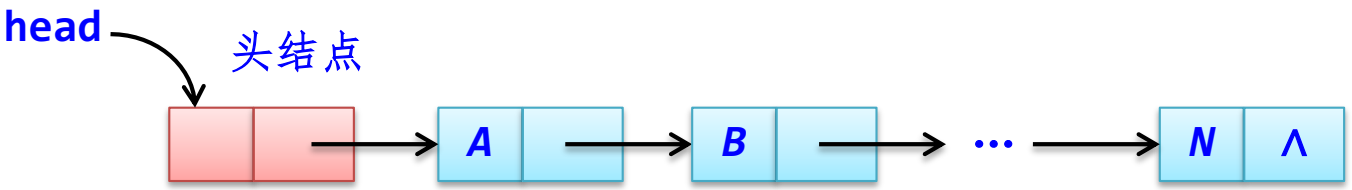
4.2.2 串的链式存储结构—链串

串的实现方式



用带头结点的单链表表示链串

例如，s= "ABCDEFGHIJKLMN"，共14个字符。



结点大小=1



结点大小=4

链串的结点类型LinkNode（结点大小为1）

```
struct LinkNode                                  //链串结点类型
{   char data;                                   //存放一个字符
    LinkNode* next;                              //指向下一个结点

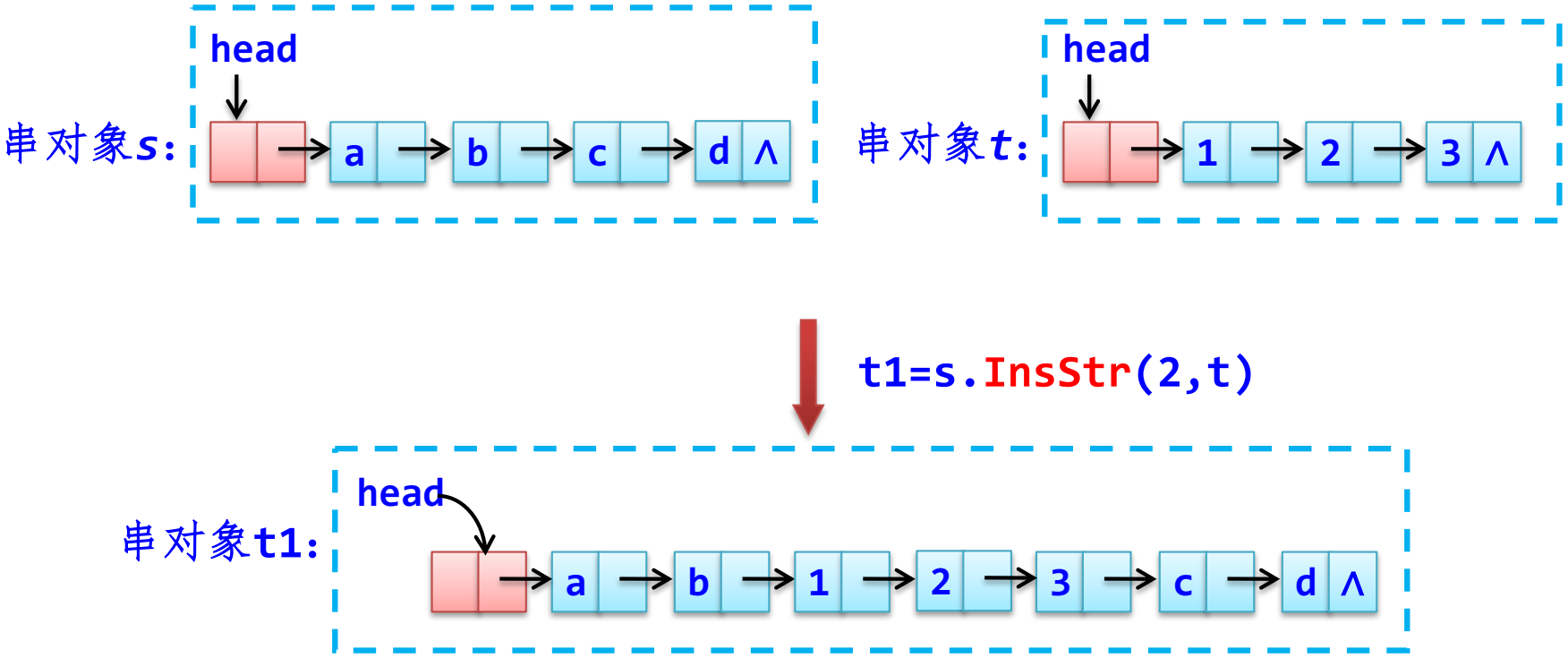
    LinkNode():next(NULL) {}                     //构造函数
    LinkNode(char d):data(d),next(NULL) {}       //重载构造函数
};
```

一个链串用一个头结点head来唯一标识，链串类LinkString

```
class LinkString                    //链串类
{
public:                             //为了简单将成员均设置为公有的
    LinkNode* head;                 //链串头结点head
    int length;                     //链串长度
  //串的基本运算算法
};
```

链串上的基本运算算法设计与单链表类似，仅以串插入算法为例说明。

串插入：链串在序号i位置插入串t





实现：

先创建一个空串s，当参数正确时，采用尾插法建立结果串s：

（1）将当前链串的前i个结点复制到s中。

（2）将t中所有结点复制到s中。

（3）再将当前串的余下结点复制到s中。

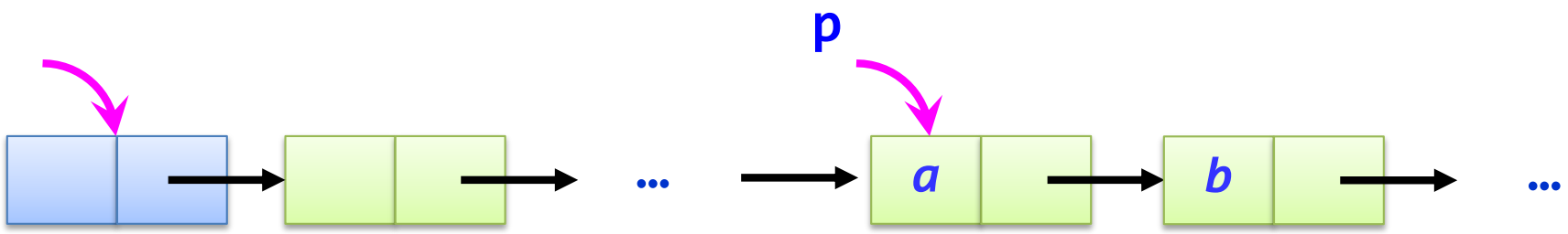
```
LinkString& InsStr(int i,LinkString& t)      //串插入
{  static LinkString s;                      //新建一个空串
   if (i<0 || i>length) return s;    //参数不正确时返回空串
   LinkNode* p=head->next,*p1=t.head->next;
   LinkNode* r=s.head,*q;            //r指向新建链表的尾结点
   for (int k=0; k<i; k++)           //将当前链串的前i个结点复制到s
   {  q=new LinkNode(p->data);       //新建结点q
      r->next=q; r=q;                //将结点q链接到尾部
      p=p->next;
   }
```

```
    while (p1!=NULL)                    //将t中所有结点复制到s
    {  q=new LinkNode(p1->data);        //新建结点q
       r->next=q; r=q;                  //将结点q链接到尾部
       p1=p1->next;
    }
    while (p!=NULL)                     //将p及其后的结点复制到s
    {  q=new LinkNode(p->data);         //新建结点q
       r->next=q; r=q;                  //将结点q链接到尾部
       p=p->next;
    }

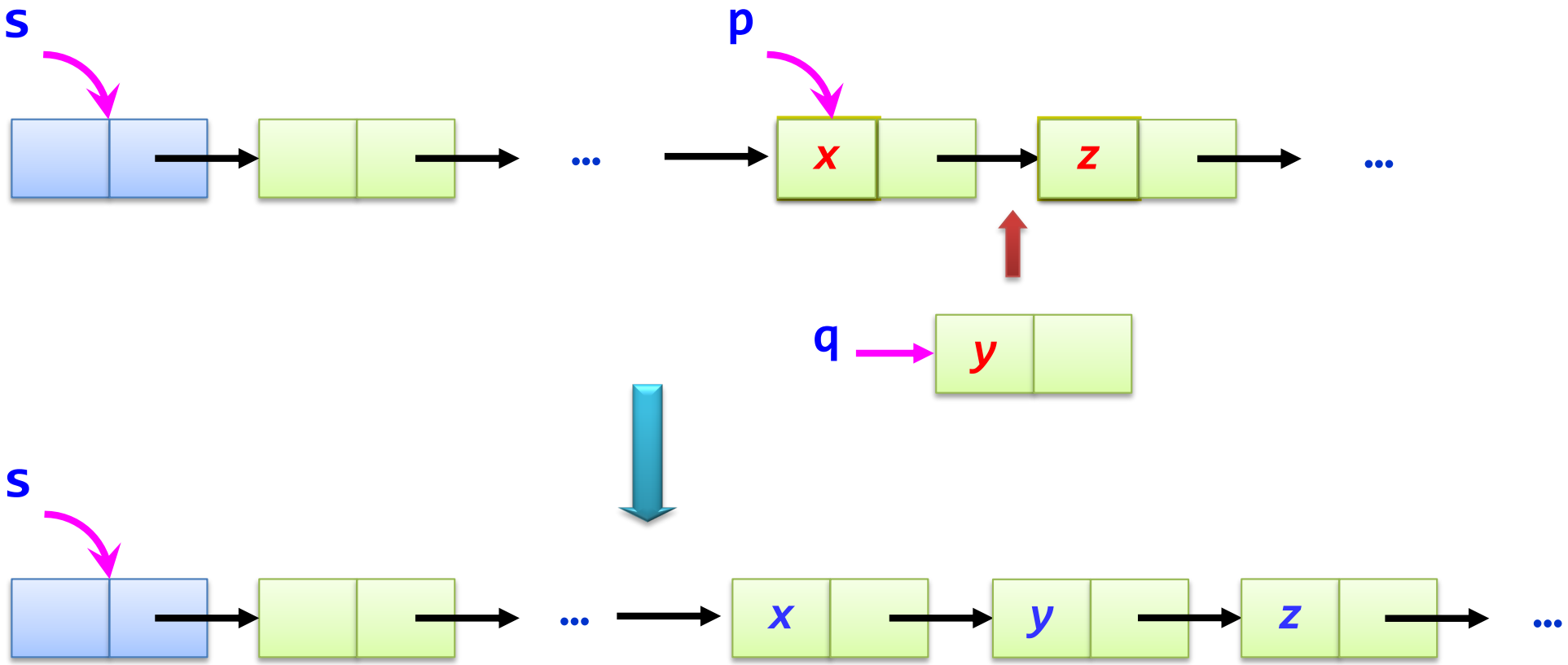
    s.length=length+t.length;
    r->next=NULL;                       //尾结点的next置为空
    return s;                           //返回新建的链串
}
```

【例4-3】在链串中，设计一个算法把最先出现的子串“*ab*”改为“*xyz*”。

❶ **查找**：`p->data='a' && p->next->data='b'`



❷ 替换



```
void Repl(LinkStrNode *&s)
{
    LinkStrNode *p=s->next, *q;
    int find=0;
    while (p->next!=NULL && find==0)          //查找ab子串
    {
        if (p->data=='a' && p->next->data=='b')
        {  p->data='x'; p->next->data='z';
           q=(LinkStrNode *)malloc(sizeof(LinkStrNode));
           q->data='y';  q->next=p->next;  p->next=q;
           find=1;
        }
        else p=p->next;
    }
}
```

替换为xyz

算法的时间复杂度为O(*n*)。

4.3 STL中的string

- STL中的string是一个保存字符序列的容器，类似vector<char>。
- string除了提供字符串的一些常用操作以外，还包含序列容器的操作。
- string的常用操作包括增加、删除、修改、查找比较、连接、输入和输出等。
- string重载了许多运算符，包括+，+=，<，=，[]，<<和>>等。

创建string容器的几种方式

创建方式	说明
string()	建立一个空的字符串
string(const string& s)	用字符串s建立当前字符串
string(conststring& str, size_t str_idx)	用字符串str起始于str_idx的字符建立当前字符串
string(const string& s,size_t i,size_t num)	用字符串s起始于i的num个字符建立当前字符串
string(const char* cstr)	用C-字符串cstr建立当前字符串
string(const char* chars,size_t len)	用C-字符串cstr的开头num个字符建立当前字符串
string(size_t num,char c)	用num个字符c建立当前字符串

string容量的主要成员函数及其说明

成员函数	说明
empty()	判断当前字符串是否为空串
length()、size()	返回当前字符串的实际字符个数（返回结果为size_t类型）
[p]	返回当前字符串位于序号p的字符，序号i从0开始。
front()	返回当前字符串的首字符
back()	返回当前字符串的尾字符
compare(const string& s)	返回当前字符串与字符串s的比较结果，若两者相等返回0，若前者小于后者返回-1，否则返回1
append(s)	在当前字符串的末尾添加一个字符串s
push_back(char c)	在当前字符串的末尾添加一个字符c
pop_back()	在当前字符串中删除末尾字符
insert(size_t p, const string& s)	在当前字符串的序号p处插入一个字符串s
insert(IT p,char c)	在当前字符串中迭代器p指向的位置上插入一个字符c

find(string& s,size_t p=0)	在当前字符串中p位置开始查找字符串s的第一个位置，找到后返回其位置，没有找到返回-1
find(char c,size_t p=0)	在当前字符串中p位置开始查找字符c的第一个位置，找到后返回其位置，没有找到返回-1
rfind(string& s,size_t p=0)	在当前字符串中p位置开始查找字符串s的最后一个位置，找到后返回其位置，没有找到返回-1
rfind(char c,size_t p=0)	在当前字符串中p位置开始查找字符c的最后一个位置，找到后返回其位置，没有找到返回-1
replace(size_t p, size_t len,const string& s)	将当前字符串中起始于p的len个字符用一个字符串s替换
replace(IT beg, IT end,const string& s)	将当前字符串中[beg,end)迭代器区间（不含end）的所有字符用字符串s替换
substr(size_t p)	返回当前字符串起始于p的子串
substr(size_t p,size_t len)	返回当前字符串起始于p的长度为len的子串

clear()	删除当前字符串中的所有的字符
erase()	删除当前字符串中的所有的字符
erase(size_t p)	删除当前字符串的从序号p开始的所有字符
erase(size_t p,size_t len)	删除当前字符串的从序号p开始的len个字符
erase(IT beg,IT end)	删除当前字符串中[beg,end)迭代器区间的所有字符
begin()	返回容器中第一个元素的迭代器
end()	返回容器中尾元素后面一个位置的迭代器
rbegin()	返回容器中尾元素的反向迭代器
rend()	返回容器中首元素前面一个位置的反向迭代器

可以使用**getline**函数从**cin**流中提取若干字符存放到**string**中。

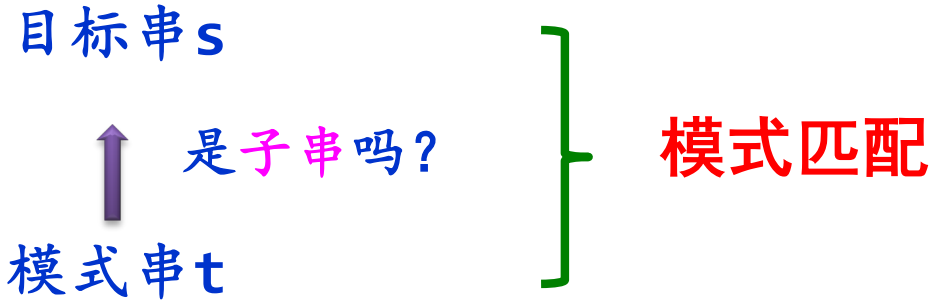
```
#include<iostream>
#include<algorithm>
using namespace std;
int main()
{  string s1="",s2,s3="Bye";
   s1.append("Good morning");          //s1="Good morning"
   s2=s1;                              //s2="Good morning"
   int i=s2.find("morning");           //i=5
   s2.replace(i,s2.length()-i,s3);     //相当于s2.replace(5,7,s3)
   cout << "s1: " << s1 << endl;
   cout << "s2: " << s2 << endl;
   cout << "s3: " << s3 << endl;
   string s4=s1;
   reverse(s4.begin(),s4.begin()+4); //逆置s4的前面4个字符
   cout << "s4: " << s4 << endl;
   return 0;
}
```



```
s1: Good morning
s2: Good Bye
s3: Bye
s4: dooG morning
```

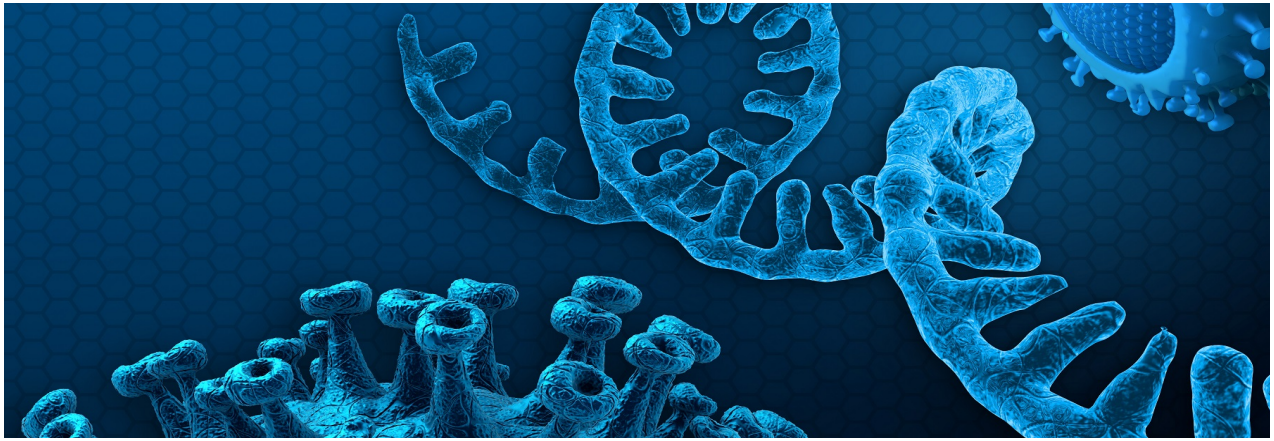
4.4 串的模式匹配

- 设有两个串s和t，串t定位操作就是在串s中查找与子串t相等的子串。
- 通常把串s称为目标串，把串t称为模式串，因此定位也称作模式匹配。
- 模式匹配**成功**是指在目标串s中找到一个模式串t。
- **不成功**则指目标串s中不存在模式串t。



应用场景：基因测序

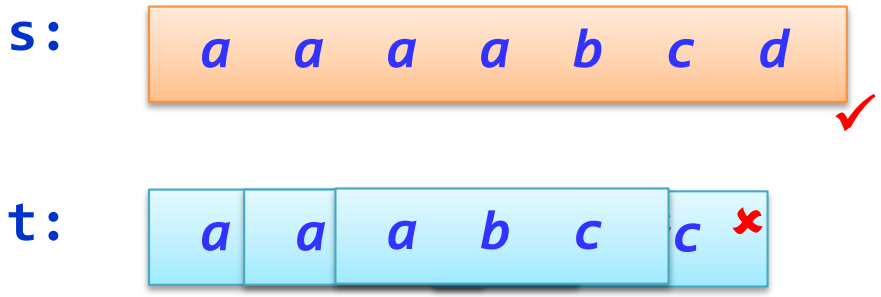
- 示例场景：生物信息学中已知一串复杂的病毒RNA序列，其规模可以达到百万级别。致病性的RNA序列段规模达到万级，此时科学家需要判断这串病毒的RNA序列中是否包含致病序列段。



- 解决方法：将复杂的病毒RNA序列认为是目标串s，致病性的RNA序列段认为是模式串t，使用字符串匹配算法（如KMP算法）解决模式匹配问题，判断模式串t是否为目标串s的子串，如果是，则可以认定该病毒具有致病性，反之则认为该病毒没有致病性。

4.4.1 BF算法

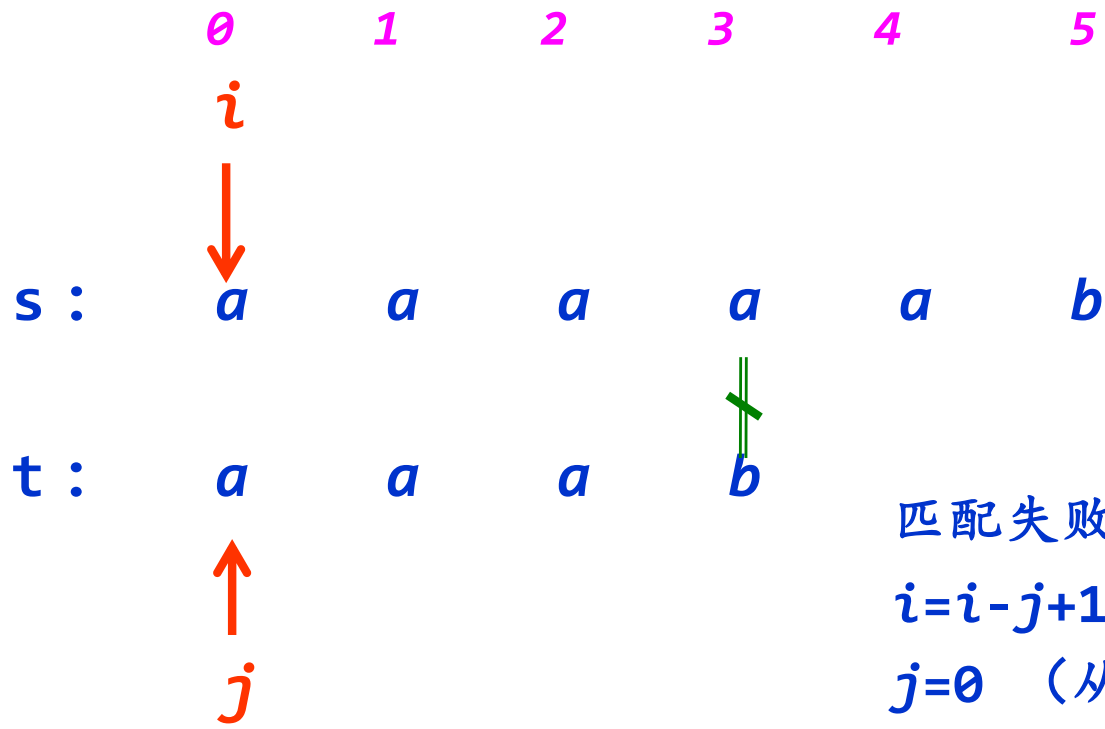
Brute-Force简称为BF算法，亦称简单匹配算法。采用穷举的思路。BF是指暴力的意思！



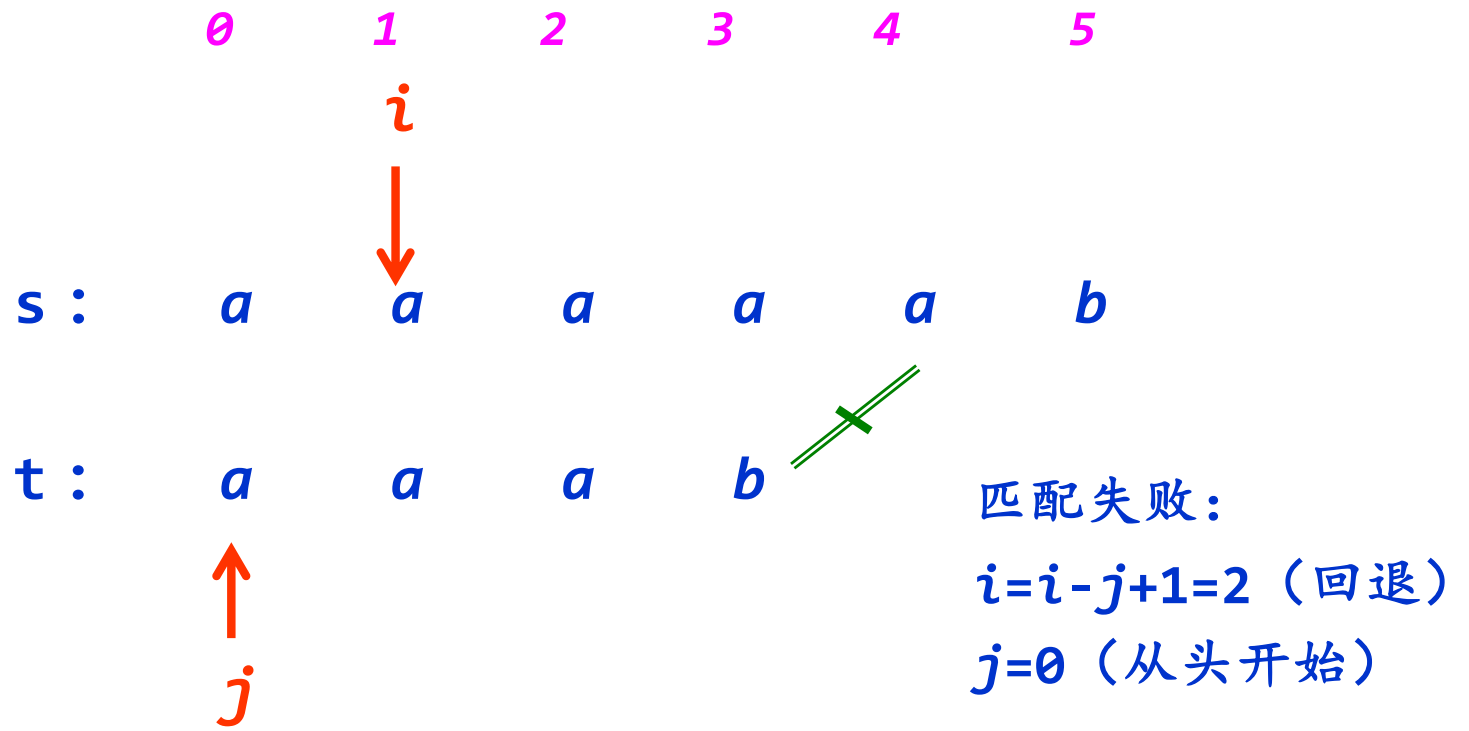
匹配成功

算法的思路是从s的每一个字符开始依次与t的字符进行匹配。

例如，设目标串s="aaaaab"，模式串t="aaab"。s的长度为n（n=6），t的长度为m（m=4）。BF算法的匹配过程如下。



i=1, j=0

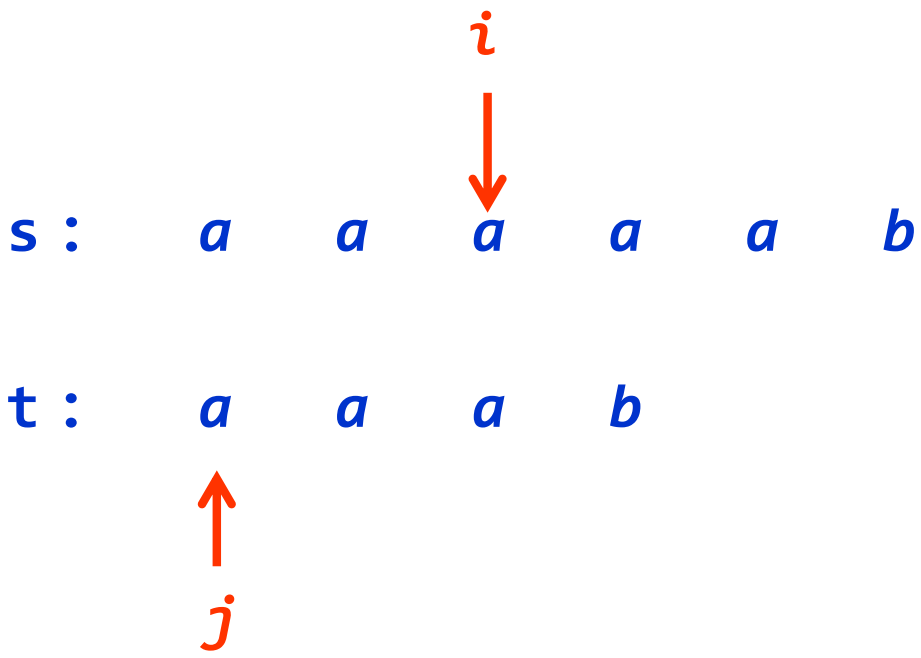


匹配失败：

i=i-j+1=2（回退）

j=0（从头开始）

i=2，j=0



匹配成功：

i=6，j=4

返回i-t.length=2

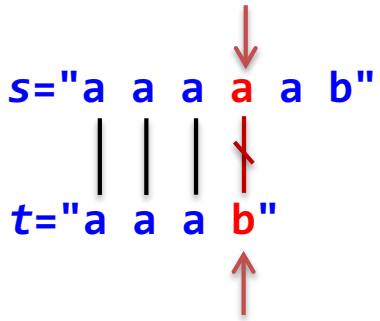
BF算法思路

目标串s="$s_0s_1\cdots s_{n-1}$"，模式串t="$t_0t_1\cdots t_{m-1}$"

- 第1趟：从s_0/t_0开始比较，若相等，则继续逐个比较后续字符。如果对应的字符全部相同且t的字符比较完，说明t是s的子串，返回t在s中的起始位置，表示匹配成功；如果对应的字符不相同，说明第一趟匹配失败。
- 第2趟：从s_1/t_0开始比较，若相等，则继续逐个比较后续字符。如果对应的字符全部相同且t的字符比较完，说明t是s的子串，返回t在s中的起始位置，表示匹配成功；如果对应的字符不相同，说明第一趟匹配失败。
- 依次类推。只要有一趟匹配成功，则说明t是s的子串，返回t在s中的起始位置。如果i超界都没有匹配成功，说明t不是s的子串，返回-1。

例如，设目标串s="aaaaab"，模式串t="aaab"

第1趟匹配



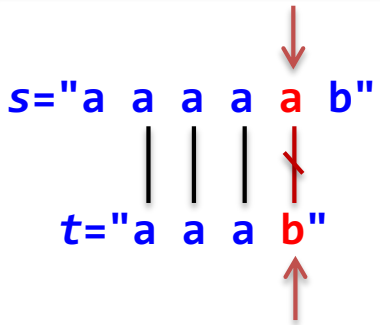
i=3

j=3

失败,修改为 → i=i-j+1=1, j=0

比较4次

第2趟匹配



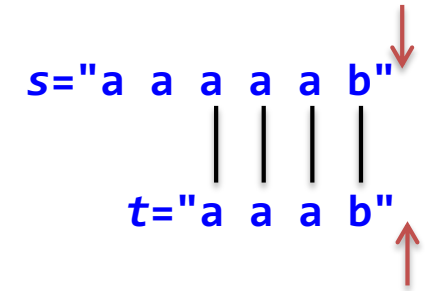
i=4

j=3

失败,修改为 → i=i-j+1=2, j=0

比较4次

第3趟匹配



i=6

j=4

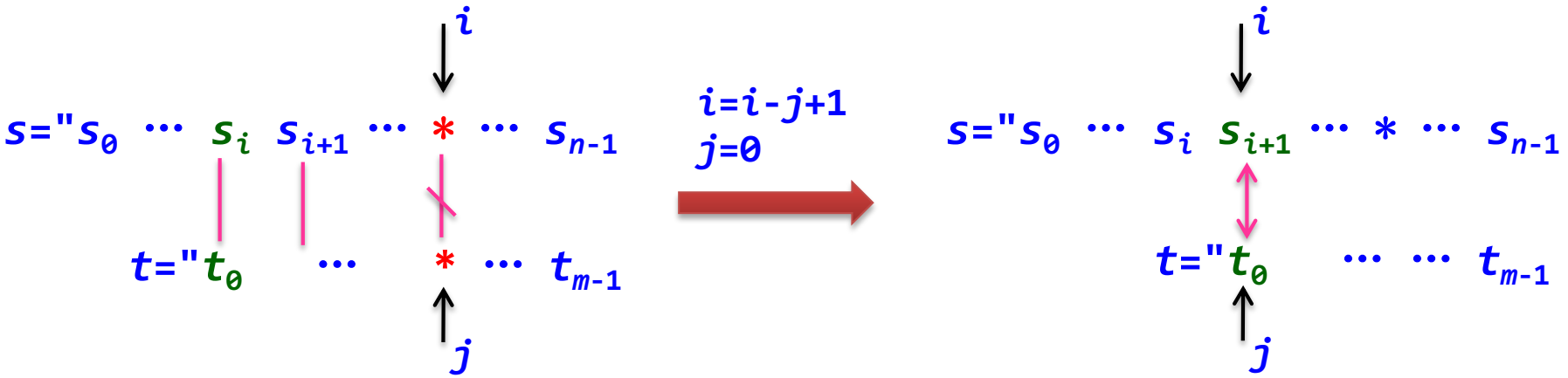
成功，返回i-t.length()=2

比较4次

共比较12次

```
int BF(string s,string t)                          //BF模式匹配算法
{  int i=0,j=0;
   while (i<s.length() && j<t.length()) //两串未遍历完时循环
   {  if (s[i]==t[j])                              //比较的两个字符相同时
      {  i++; j++; }                               //s和t串依次移到下一个字符
      else                                         //比较的两个字符不相同时
      {  i=i-j+1;                                  //i回退到s本趟开始的下一个字符
         j=0;                                      //j移动到t串的开头
      }
   }

   if (j>=t.length()) return i-t.length(); //t串遍历完毕：匹配成功
   else return -1;                                 //匹配不成功，返回-1
}
```



失配处：s_i/t_j

BF算法分析 —— 最坏情况

1) *AAAAA*AAAAAAAAAAAAAAAAAAAAAAAAH
AAAAH **5 comparisons made**

2) *AAAAA*AAAAAAAAAAAAAAAAAAAAAAAAH
AAAAH **5 comparisons made**

3) *AAAAA*AAAAAAAAAAAAAAAAAAAAAAAAH
AAAAH **5 comparisons made**

4) *AAAAA*AAAAAAAAAAAAAAAAAAAAAAAAH
AAAAH **5 comparisons made**

5) *AAAAA*AAAAAAAAAAAAAAAAAAAAAAAAH
AAAAH **5 comparisons made**

....

N) AAAAAAAAAAAAAAAAAAAAAAAAAAAA*AAAAH*
5 comparisons made *AAAAH*

- 总比较次数 $m\times(n-m+1)$
- 最坏情况下的时间复杂度为$O(n\times m)$

BF算法分析 —— 最好情况

模型匹配成功

AAAAAAAAAAAAAAAAAAAAAAAAAAAAAH
AAAAA　　5 comparisons made

- 总比较次数 m
- 最好情况下的时间复杂度为$O(m)$

BF算法分析 —— 最好情况

1) *A*AAAAAAAAAAAAAAAAAAAAAAAAAAAH
*O*OOOH **1 comparison made**

2) A*A*AAAAAAAAAAAAAAAAAAAAAAAAAAH
*O*OOOH **1 comparison made**

3) AA*A*AAAAAAAAAAAAAAAAAAAAAAAAAH
*O*OOOH **1 comparison made**

4) AAA*A*AAAAAAAAAAAAAAAAAAAAAAAAH
*O*OOOH **1 comparison made**

5) AAAA*A*AAAAAAAAAAAAAAAAAAAAAAAH
*O*OOOH **1 comparison made**

...

N) AAAAAAAAAAAAAAAAAAAAAAAAAA*A*AAAH
1 comparison made *O*OOOH

- 总比较次数 *n-m+1*
- 最好情况下的时间复杂度为O(*n*)

BF算法性能

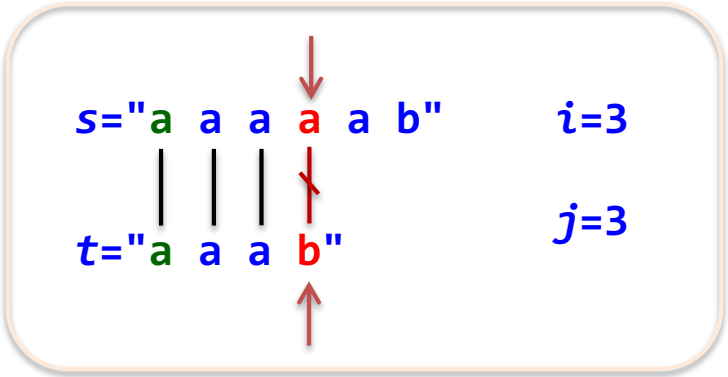
- 该算法在最好情况下的时间复杂度为$O(m)$，即主串的前m个字符正好等于模式串的m个字符。
- 最坏情况下的时间复杂度为$O(n \times m)$。
- 平均情况下的时间复杂度为$O(n \times m)$。

4.4.2 KMP算法

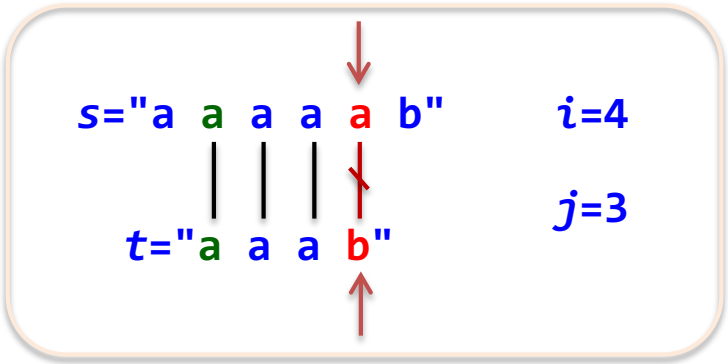
基本KMP算法

主要是消除了目标串指针的回溯，从而使算法效率有了某种程度的提高。

第1趟匹配



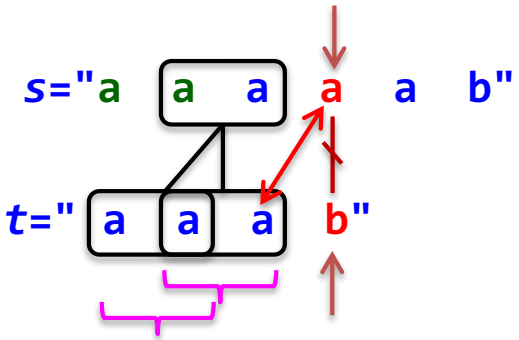
第2趟匹配



匹配失败×

失配处

第1趟匹配

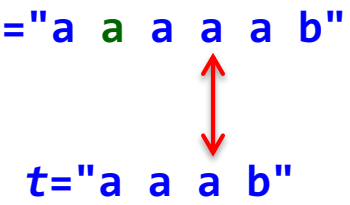


i=3

j=3

这种信息可以匹配之前获取

第2趟匹配



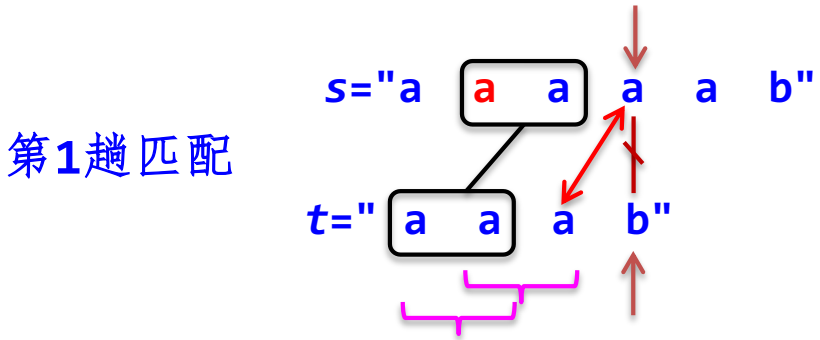
跳过第2趟匹配前面2个字符的比较

KMP 核心：

主串指针 i 永不回溯，只让模式串指针 j 回退

利用“已经匹配上的前缀信息”跳过无效比对

失配处



第1趟匹配

这种信息可以匹配之前获取

后缀　　前缀

- t_j的前面有多少个连续字符（不含t_0）和t开头的连续字符相同！
- 用next数组存放，这里next[3]=2。
- 下一次做$s_i/t_{next[j]}$比较。

字符串相关概念

- (真)前缀：从长度为n的字符串第0位开始，第i位（0≤i<n-1）结束的任意子串。对字符串s，其前缀可表示为s[0...i](0≤i<n-1)。字符串的所有前缀构成的集合称为前缀集合。前缀不包含最后一个字符
- (真)后缀：从长度为n的字符串第i位（0<i ≤n-1）开始，最后一位结束的任意子串。对字符串s，其后缀可表示为s[i...n-1](0<i≤n-1)。字符串的所有后缀构成的集合称为后缀集合。后缀不包含第一个字符
- 公共(真)前后缀：字符串的前缀集合与后缀集合中相同的子串。
- 最长(真)公共前后缀：字符串的前缀集合与后缀集合中相同的长度最长的子串。

例：字符串"aabaa“
前缀集合为{"a","aa","aab","aaba"}
后缀集合为{"a","aa","baa","abaa"}
其公共前后缀包括"a"和"aa"两个子串
最长公共前后缀为"aa"这个子串。

next[j]是指t[j]字符前有多少个字符与t开头的字符相同。

next[j]是t的子串t[0...j-1]的最长公共前后缀的长度。

模式串t存在某个k（0<k<j），使得以下成立：

“$t_0t_1\cdots t_{k-1}$” = “ $t_{j-k}t_{j-k+1}\cdots t_{j-1}$ ”

开头的k个字符　　t[j]前面的k个字符

0 1 2 3 4

例如，t= “a b a b c” 考虑t[4]='c'



有t_0t_1= t_2t_3 = "ab" ⇨ k=2

所以next[4] = k = 2。

next[j]的作用是：指当模式串的第 j 位和主串失配时，
j 不用归零，直接回退到next[j]的位置继续比对。

归纳起来，求模式t的next[j]（$0 \leqslant j \leqslant m-1$）数组的公式如下：

后缀不含t_j，同时$j-k \geqslant 1$，即后缀至多从t_1开始而不能从t_0开始

$$
next[j]=\begin{cases} MAX\{k \mid 0<k<j \text{且} "t_0t_1\cdots t_{k-1}"="t_{j-k}t_{j-k+1}\cdots t_{j-1}"\} & \text{当前缀非空时} \\ -1 & \text{当}j=0\text{时} \\ 0 & \text{其他情况} \end{cases}
$$

t_{j-1}前缀　　t_{j-1}的后缀

均含k个字符

模式串t="abcac"

j	t[j]	t[0..j-1] 子串	前缀	后缀	最长公共 前后缀	next[j]
0	a					-1
1	b	a				0
2	c	ab	a	b		0
3	a	abc	a,ab	c,bc		0
4	c	abca	a,ab,abc	a,ca,bca	a	1

任何字符串都成立

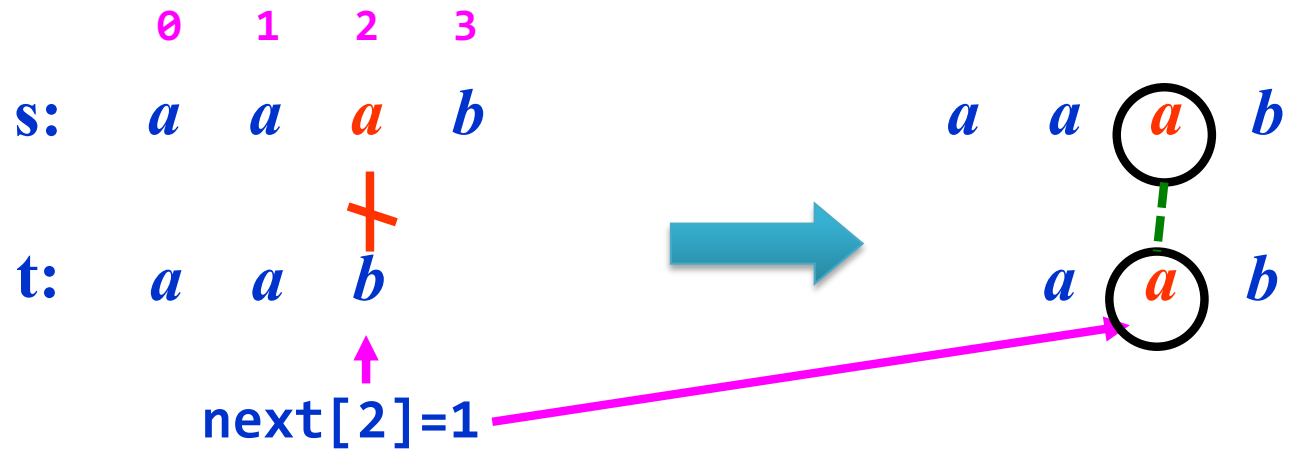


j	0	1	2	3	4
t[j]	a	b	c	a	c
next[j]	-1	0	0	0	1

next[j]的含义

（1）next[j]=k表示什么信息？

说明模式串t[j]之前有k个字符已成功匹配，下一趟应从t[k]开始匹配。



（2）next[j] = 0 表示什么信息？

说明模式串t[j]之前没有任何用于加速匹配的信息，下一趟应从t的开头即j=0开始匹配。

如t=“abcd”，next[1]=next[2]=next[3]= 0。

（3）next[0] = -1 表示什么信息？

说明模式串t中已经没有字符当前字符s_0进行比较了，i应该移到目标串s的下一个字符，再和模式串t的第一个字符进行比较。

KMP算法的匹配过程(一般情况)

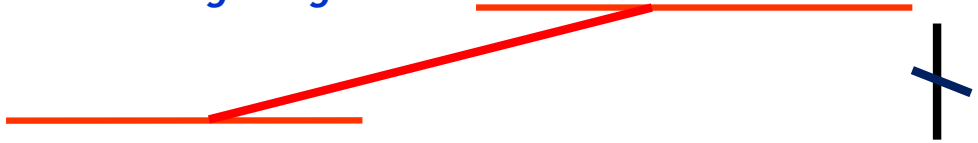
目标串s: $s_0 s_1 \cdots s_{i-j} s_{i-j+1} \cdots s_{i-k} s_{i-k+1} \cdots s_{i-1} s_i \cdots s_n$

部分匹配，k个字符相同

模式串t: $t_0 \cdots\cdots t_{k-1} t_k \cdots t_{j-k} t_{j-k+1} \cdots t_{j-1} t_j \cdots t_m$

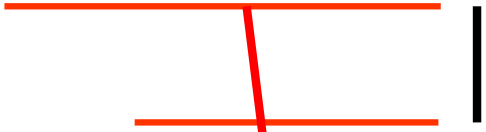


目标串s: $s_0 s_1 \cdots s_{i-j} s_{i-j+1} \cdots s_{i-k} s_{i-k+1} \cdots s_{i-1} s_i \cdots s_n$



模式串t: $t_0 \cdots\cdots t_{k-1} t_k \cdots t_{j-k} t_{j-k+1} \cdots t_{j-1} t_j \cdots t_m$

目标串s: $s_0 s_1 \cdots s_{i-j} s_{i-j+1} \cdots s_{i-k} s_{i-k+1} \cdots s_{i-1} s_i \cdots s_n$



模式串t: $t_0 \cdots\cdots t_{k-1} t_k \cdots t_{j-k} t_{j-k+1} \cdots t_{j-1} t_j \cdots t_m$

```
int KMP(string s,string t)
{   int n=s.length(),m=t.length();
    int* next=new int[m];
    GetNext(t,next);
    int i=0,j=0;
    while (i<n && j<m)
    {   if (j==-1 || s[i]==t[j])
        {   i++;
            j++;
        }
        else j=next[j];
    }
    if (j>=m) return i-m;
    else return -1;
}
```

情况0 匹配成功

继续比较目标串和模式串下一个字符。

情况3 next[j]=-1

说明模式串t中已经没有字符当前字符s_0进行比较了，i应该移到目标串s的下一个字符，再和模式串t的第一个字符进行比较。

情况1 next[j]=k不为0

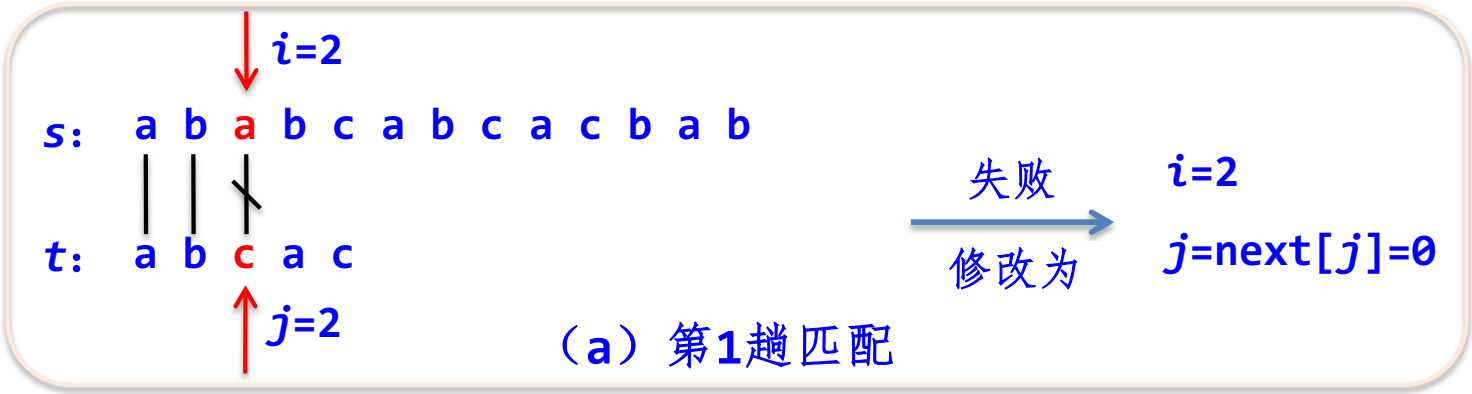
说明模式串t[j]之前有k个字符已成功匹配，下一趟应从t[k]开始匹配。

情况2 next[j]=0

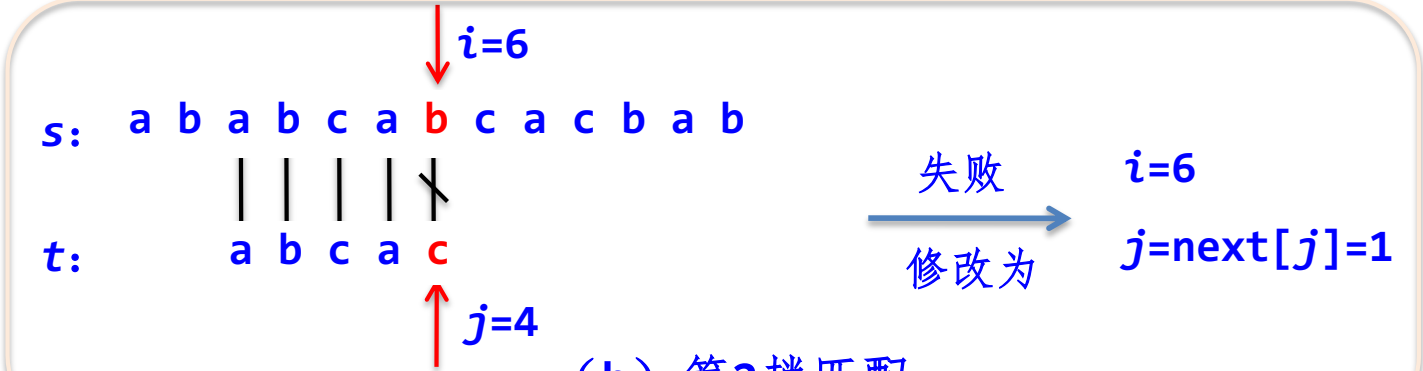
说明模式串t[j]之前没有任何用于加速匹配的信息，下一趟应从t的开头即j=0开始匹配。

【例4.5】设主串s="ababcabcacbab"，模式串t="abcac"。给出KMP进行模式匹配的过程。

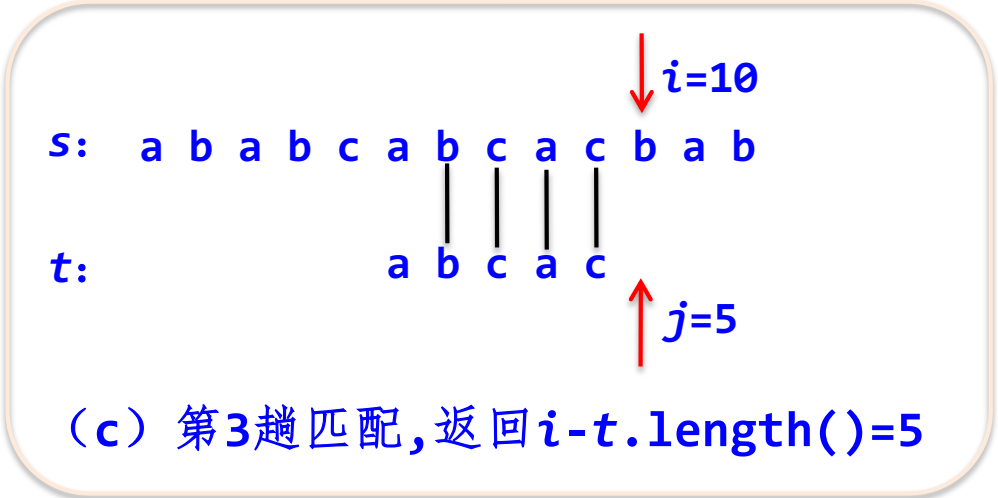
j	0	1	2	3	4
t[j]	a	b	c	a	c
next[j]	-1	0	0	0	1



（a）第1趟匹配



（b）第2趟匹配



（c）第3趟匹配,返回i-t.length()=5

KMP算法性能

- 设目标串s的长度为n，模式串t长度为m。
- 在KMP算法中求next数组的时间复杂度为$O(m)$。
- 在后面的匹配中因主串s的下标i不减即不回溯，比较次数可记为n。
- KMP算法总的平均时间复杂度为$O(n+m)$。

```
void GetNext(string t,int* next)      //由模式串t求出next值
{  int j=0, k=-1;
   next[0]=-1;
   while (j<t.length()-1)
   {  if (k==-1 || t[j]==t[k])        //k为-1或比较的字符相等时
      {  j++; k++;                    //依次移到下一个字符
         next[j]=k;
      }
      else k=next[k];                 //比较的字符不相等时k回退
   }
}
```

```
void GetNext(string t,int* next)      //由模式串t求出next值
{  int j=0, k=-1;
   next[0]=-1;
   while (j<t.length()-1)
   {  if (k==-1 || t[j]==t[k])        //k为-1或比较的字符相等时
      {  j++; k++;                    //依次移到下一个字符
         next[j]=k;
      }
      else k=next[k];                 //比较的字符不相等时k回退
   }
}
```



next[*j*] ⇨ next[*j*+1]

由模式串t求next值的算法：

```
void GetNext(SqString t, int next[])
{  int j, k;
   j=0;  k=-1;
   next[0]=-1;

   while (j<t.length-1)
   {  if (k==-1||t.data[j]==t.data[k])
      {    j++;
           k++;
           next[j]=k;
      }
      else  k=next[k];   //k回退
   }
}
```

Next数组的计算，本质就是
“模式串自己和自己做 KMP 匹配”

KMP算法：

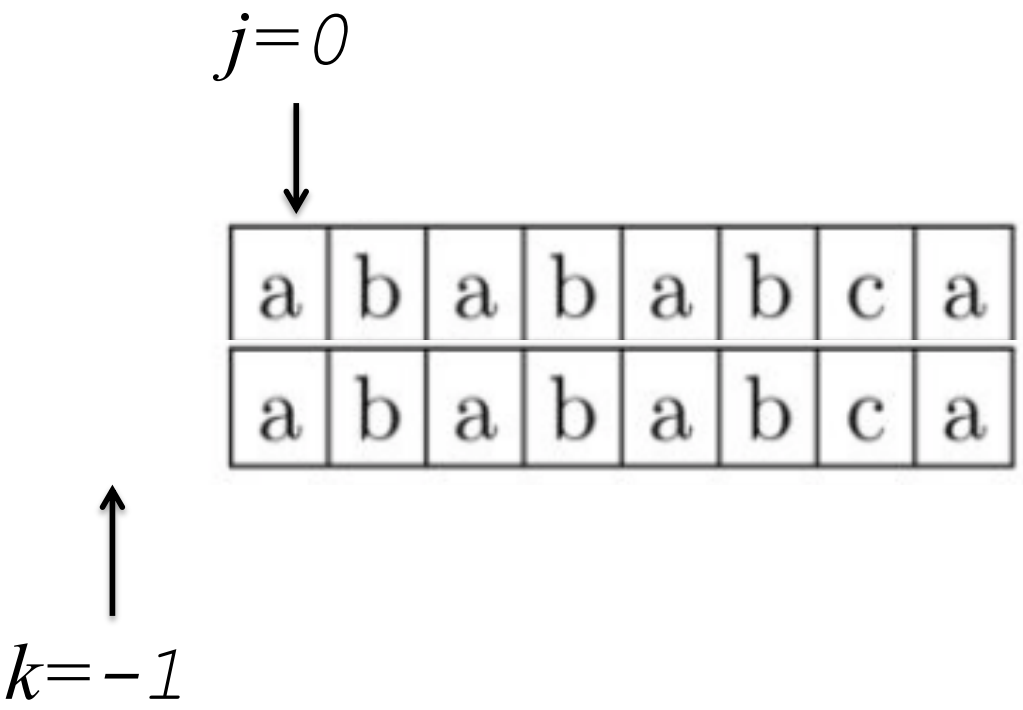
```
int KMP(SqString s, SqString t)
{  int next[MaxSize],  i=0,  j=0;
   GetNext(t, next);

   while (i<s.length && j<t.length)
   {  if (j==-1||s.data[i]==t.data[j])
        {   i++;
            j++;

        }
        else  j=next[j];
    }
    if (j>=t.length)
        return(i-t.length);
    else
        return(-1);
}
```

next数组求解过程

next[0]= -1

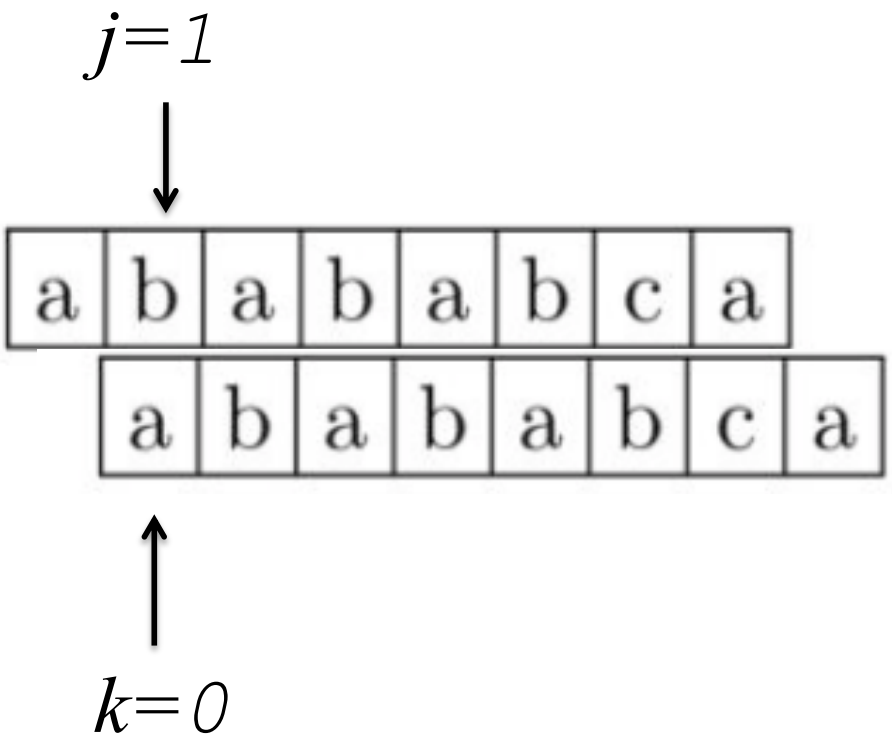



```
void GetNext(SqString t, int next[])
{  int j, k;
   j=0;  k=-1;
   next[0]=-1;

   while (j<t.length-1)
   {  if (k==-1||t.data[j]==t.data[k])
      {    j++;
           k++;
           next[j]=k;
      }
      else  k=next[k];   //k回退
   }
}
```

next数组求解过程

next[0]= -1 next[1]= 0

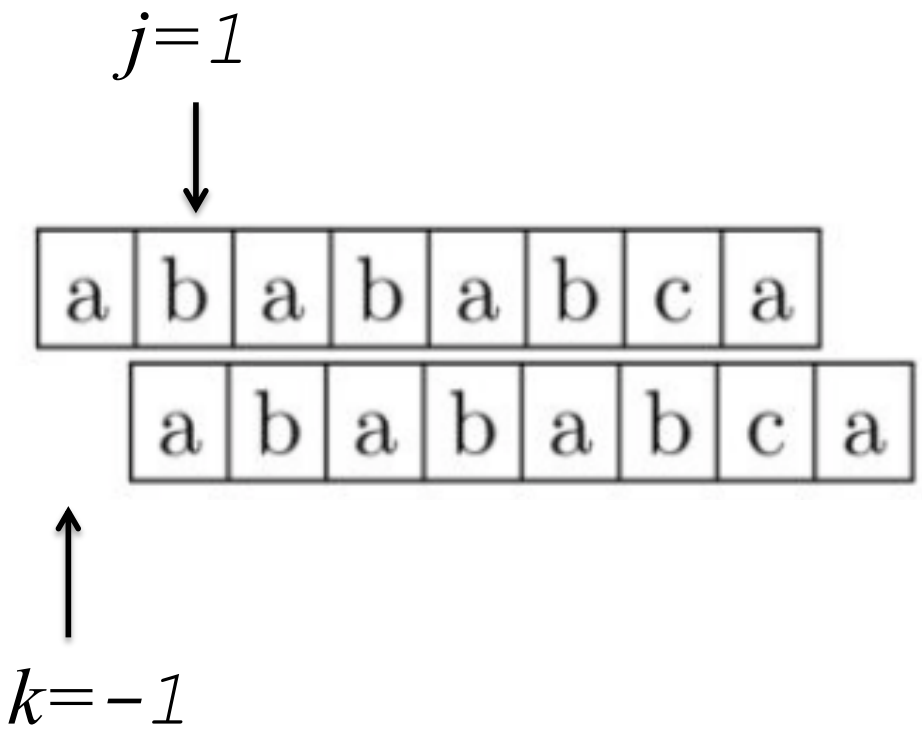



```
void GetNext(SqString t, int next[])
{  int j, k;
   j=0;  k=-1;
   next[0]=-1;

   while (j<t.length-1)
   {  if (k==-1||t.data[j]==t.data[k])
      {    j++;
           k++;
           next[j]=k;
      }
      else  k=next[k];   //k回退
   }
}
```

next数组求解过程

next[0]= -1 next[1]= 0

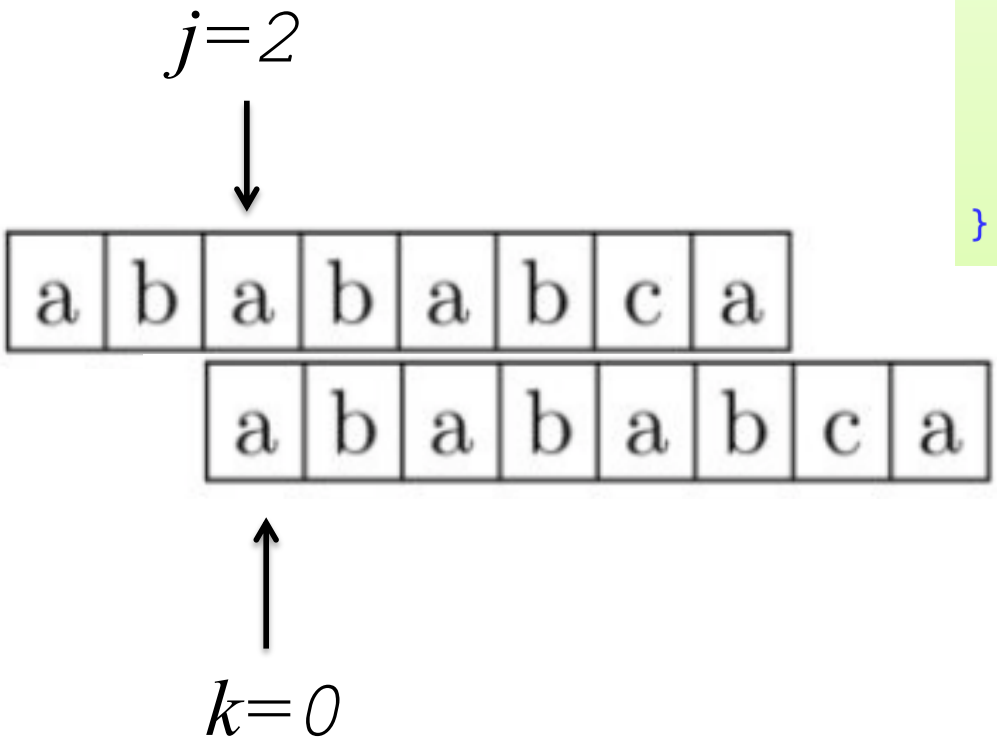



```
void GetNext(SqString t, int next[])
{  int j, k;
   j=0;  k=-1;
   next[0]=-1;

   while (j<t.length-1)
   {  if (k==-1||t.data[j]==t.data[k])
      {    j++;
           k++;
           next[j]=k;
      }
      else  k=next[k];   //k回退
   }
}
```

next数组求解过程

next[0]= -1 next[1]= 0 next[2]= 0

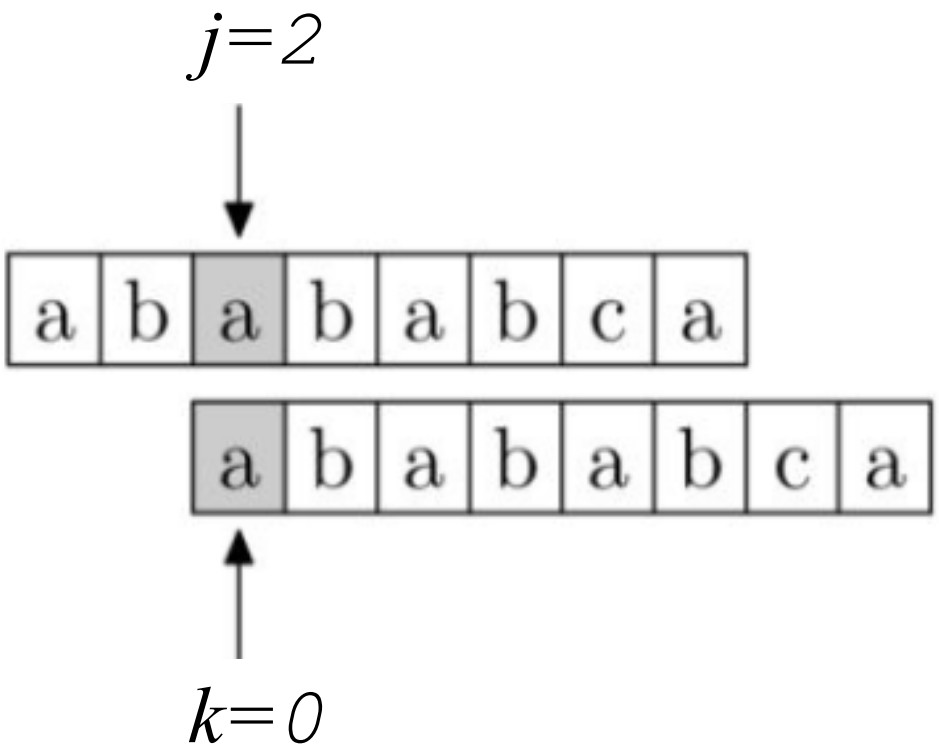



```
void GetNext(SqString t, int next[])
{  int j, k;
   j=0;  k=-1;
   next[0]=-1;

   while (j<t.length-1)
   {  if (k==-1||t.data[j]==t.data[k])
      {    j++;
           k++;
           next[j]=k;
      }
      else  k=next[k];   //k回退
   }
}
```

next数组求解过程

next[0]= -1 next[1]= 0 next[2]= 0




```
void GetNext(SqString t, int next[])
{  int j, k;
   j=0;  k=-1;
   next[0]=-1;

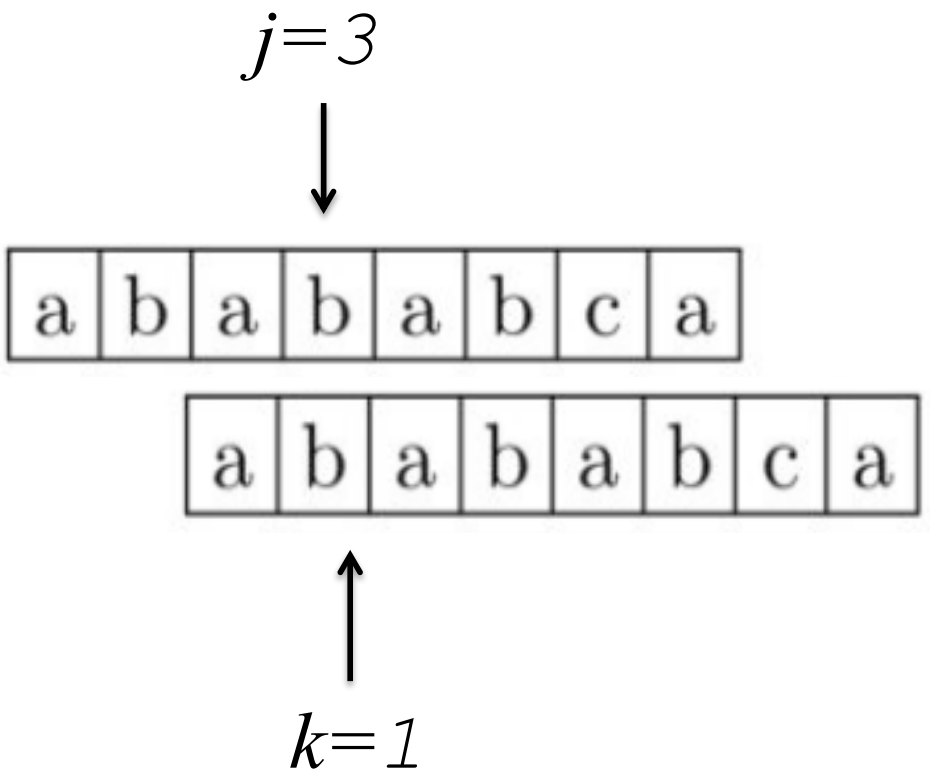
   while (j<t.length-1)
   {  if (k==-1||t.data[j]==t.data[k])
      {    j++;
           k++;
           next[j]=k;
      }
      else  k=next[k];   //k回退
   }
}
```

next数组求解过程

```
void GetNext(SqString t, int next[])
{  int j, k;
   j=0;  k=-1;
   next[0]=-1;

   while (j<t.length-1)
   {  if (k==-1||t.data[j]==t.data[k])
      {    j++;
           k++;
           next[j]=k;
      }
      else  k=next[k];   //k回退
   }
}
```

next[0]= -1 next[1]= 0 next[2]= 0



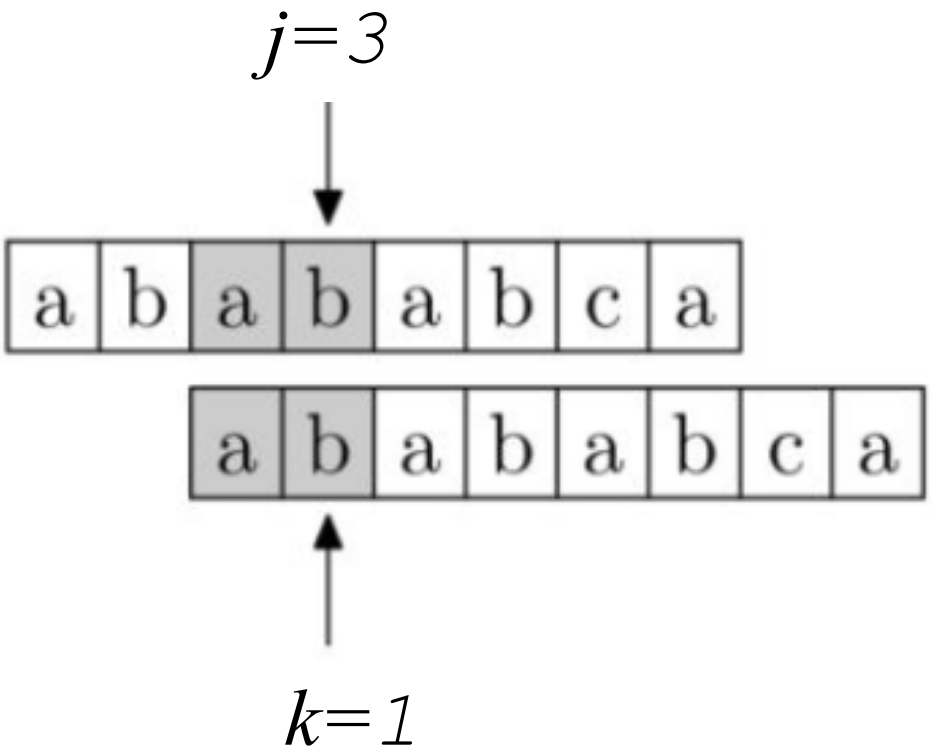
next[3]= 1

next数组求解过程

```
void GetNext(SqString t, int next[])
{  int j, k;
   j=0;  k=-1;
   next[0]=-1;

   while (j<t.length-1)
   {  if (k==-1||t.data[j]==t.data[k])
      {    j++;
           k++;
           next[j]=k;
      }
      else  k=next[k];   //k回退
   }
}
```

next[0]= -1 next[1]= 0 next[2]= 0



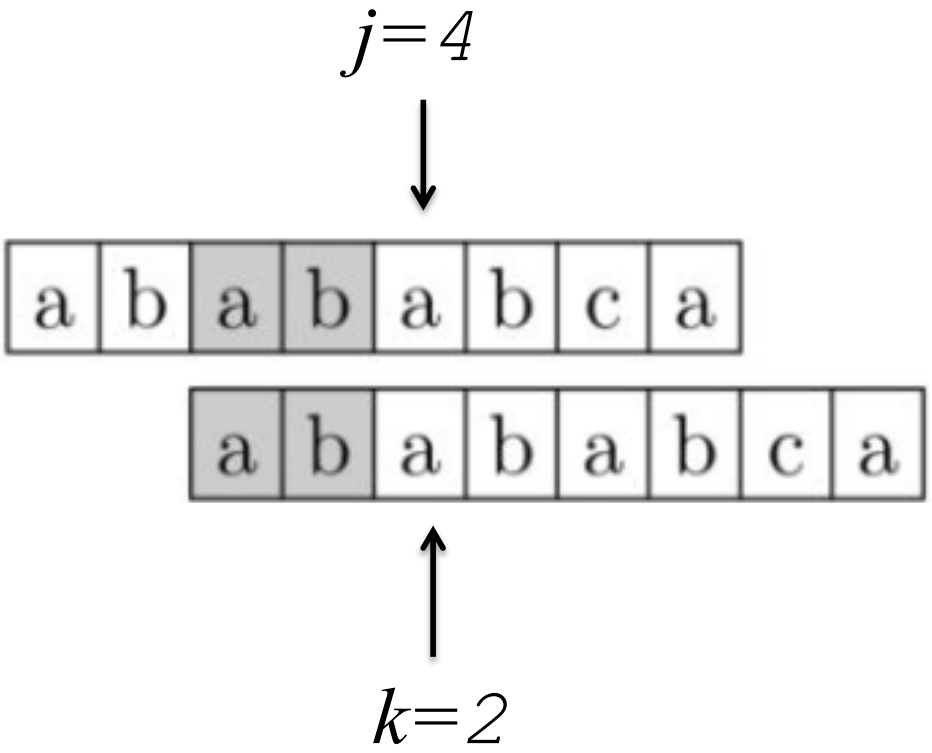
next[3]= 1

next数组求解过程

```
void GetNext(SqString t, int next[])
{  int j, k;
   j=0;  k=-1;
   next[0]=-1;

   while (j<t.length-1)
   {  if (k==-1||t.data[j]==t.data[k])
      {    j++;
           k++;
           next[j]=k;
      }
      else  k=next[k];   //k回退
   }
}
```

next[0]= -1 next[1]= 0 next[2]= 0



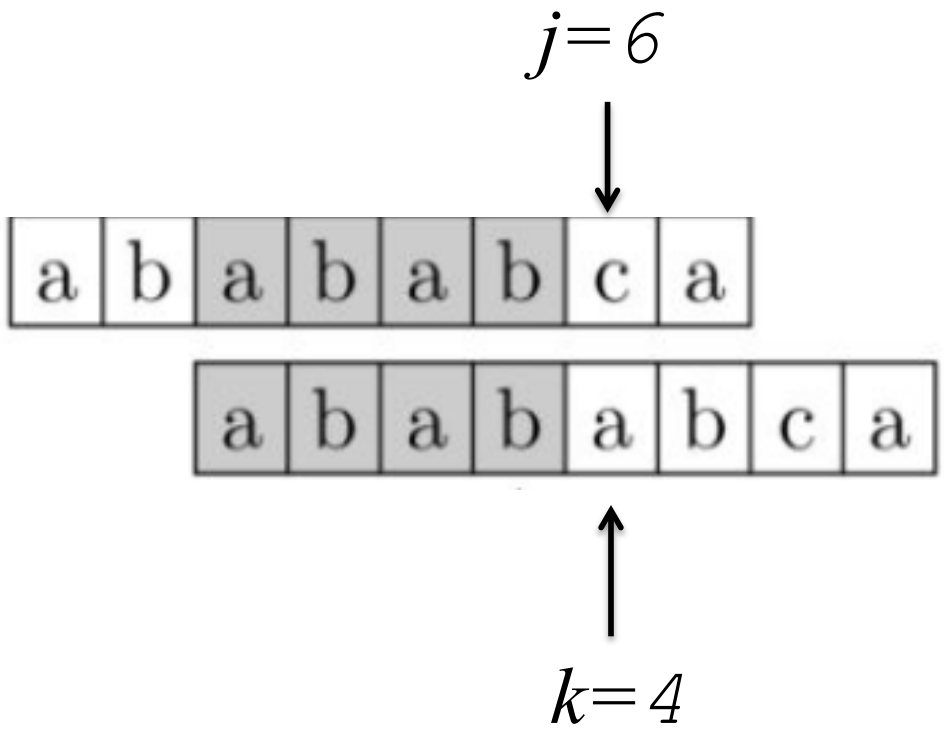
next[3]= 1 next[4]= 2

next数组求解过程

```
void GetNext(SqString t, int next[])
{  int j, k;
   j=0;  k=-1;
   next[0]=-1;

   while (j<t.length-1)
   {  if (k==-1||t.data[j]==t.data[k])
      {    j++;
           k++;
           next[j]=k;
      }
      else  k=next[k];   //k回退
   }
}
```

next[0]= -1 next[1]= 0 next[2]= 0



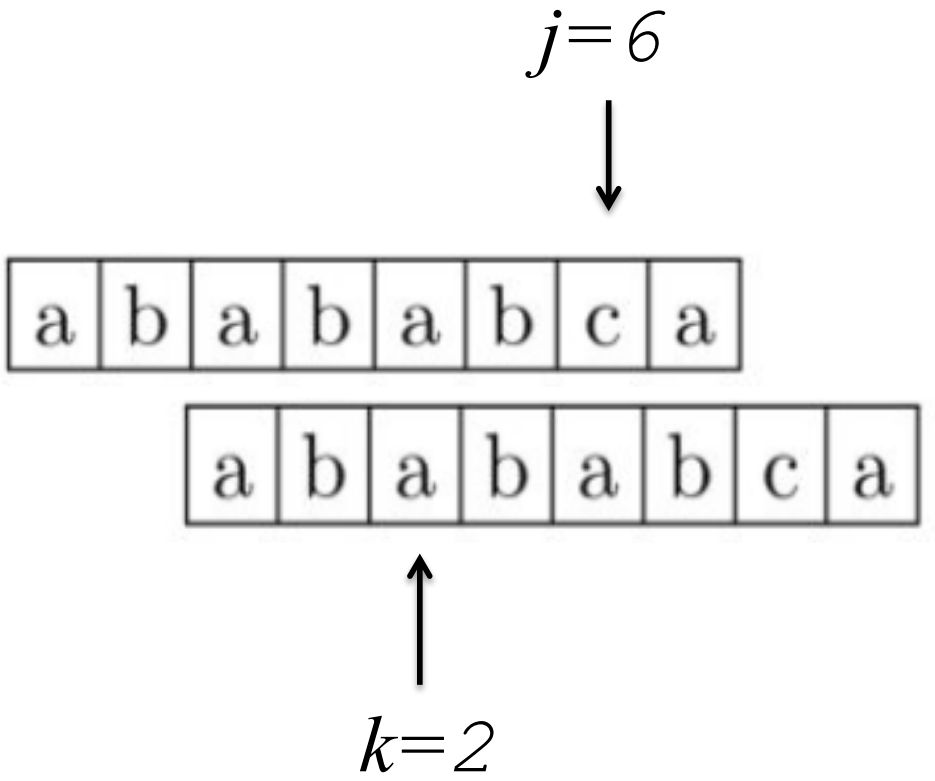
next[3]= 1 next[4]= 2 next[5]= 3 next[6]= 4

next数组求解过程

```
void GetNext(SqString t, int next[])
{  int j, k;
   j=0;  k=-1;
   next[0]=-1;

   while (j<t.length-1)
   {  if (k==-1||t.data[j]==t.data[k])
      {    j++;
           k++;
           next[j]=k;
      }
      else  k=next[k];   //k回退
   }
}
```

next[0]= -1 next[1]= 0 next[2]= 0



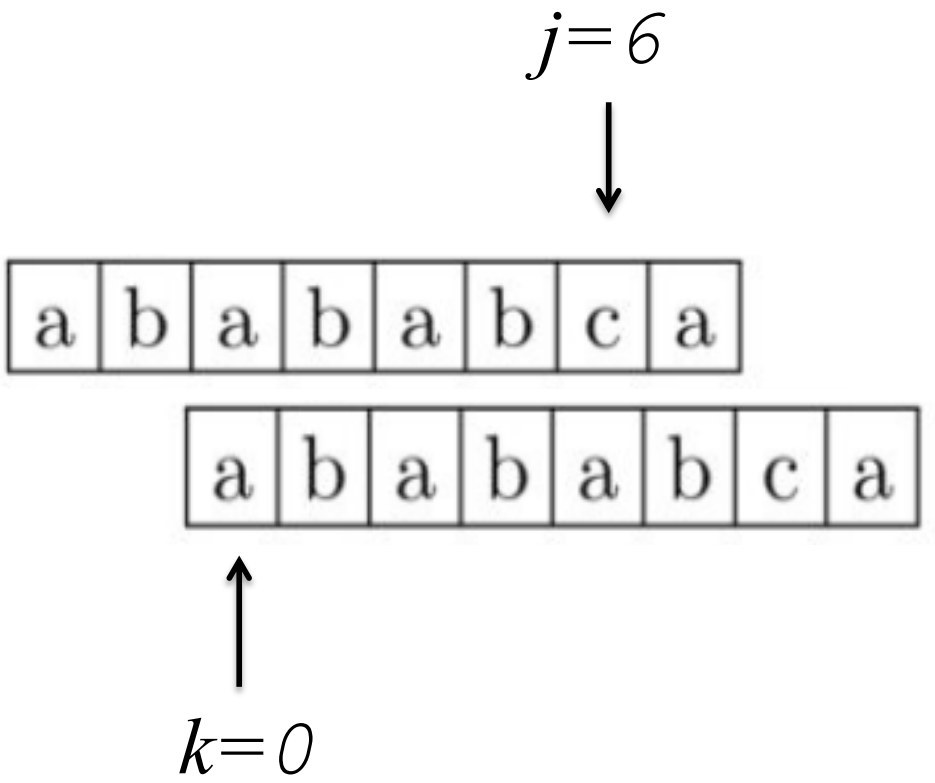
next[3]= 1 next[4]= 2 next[5]= 3 next[6]= 4

next数组求解过程

```
void GetNext(SqString t, int next[])
{  int j, k;
   j=0;  k=-1;
   next[0]=-1;

   while (j<t.length-1)
   {  if (k==-1||t.data[j]==t.data[k])
      {    j++;
           k++;
           next[j]=k;
      }
      else  k=next[k];   //k回退
   }
}
```

next[0]= -1 next[1]= 0 next[2]= 0



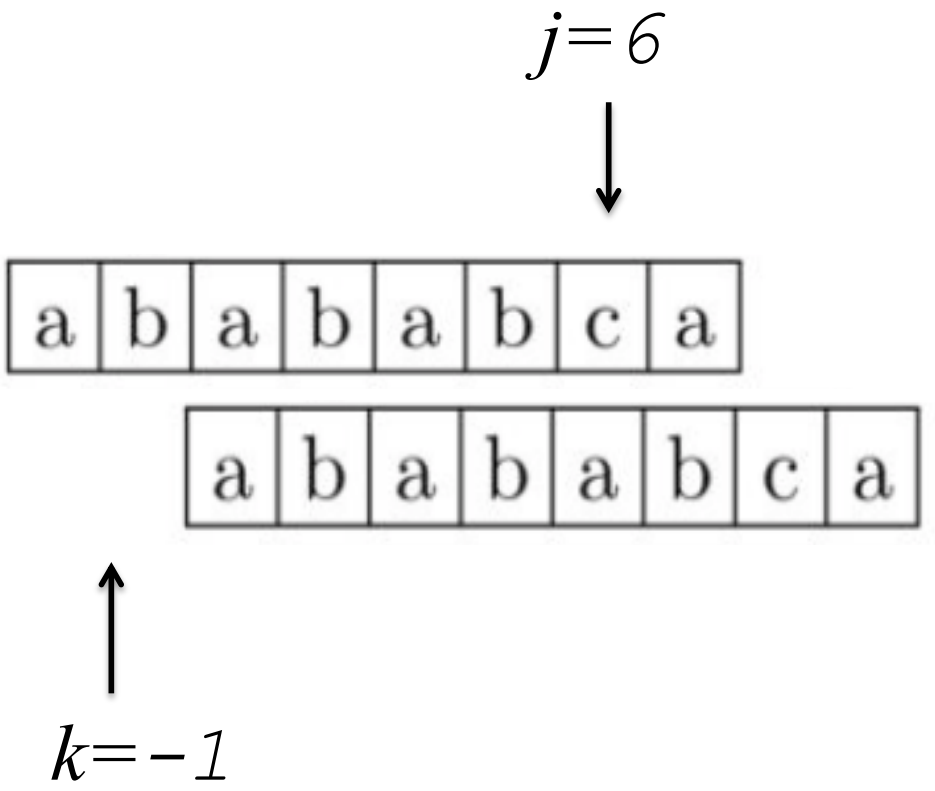
next[3]= 1 next[4]= 2 next[5]= 3 next[6]= 4

next数组求解过程

```
void GetNext(SqString t, int next[])
{  int j, k;
   j=0;  k=-1;
   next[0]=-1;

   while (j<t.length-1)
   {  if (k==-1||t.data[j]==t.data[k])
      {    j++;
           k++;
           next[j]=k;
      }
      else  k=next[k];   //k回退
   }
}
```

next[0]= -1 next[1]= 0 next[2]= 0



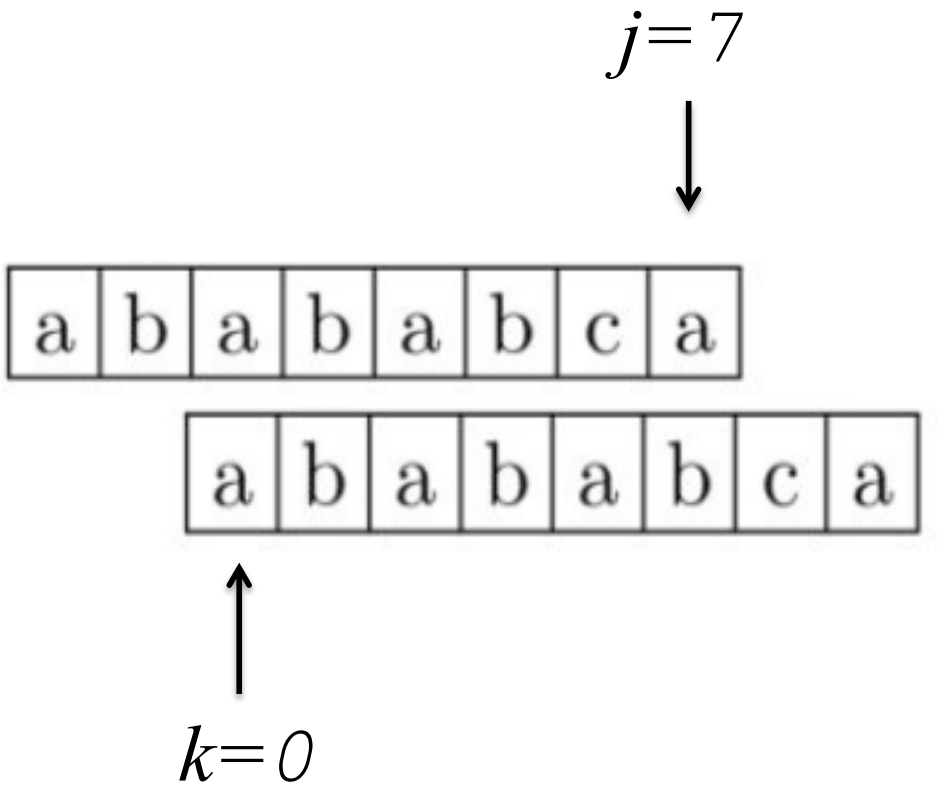
next[3]= 1 next[4]= 2 next[5]= 3 next[6]= 4

next数组求解过程

```
void GetNext(SqString t, int next[])
{  int j, k;
   j=0;  k=-1;
   next[0]=-1;

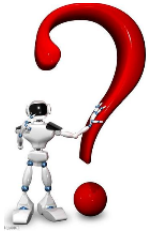
   while (j<t.length-1)
   {  if (k==-1||t.data[j]==t.data[k])
      {    j++;
           k++;
           next[j]=k;
      }
      else  k=next[k];   //k回退
   }
}
```

next[0]= -1 next[1]= 0 next[2]= 0



next[3]= 1 next[4]= 2 next[5]= 3 next[6]= 4

next[7]= 0



- KMP算法的性能提高了吗?
- KMP算法跳过了中间一些趟，正确吗?

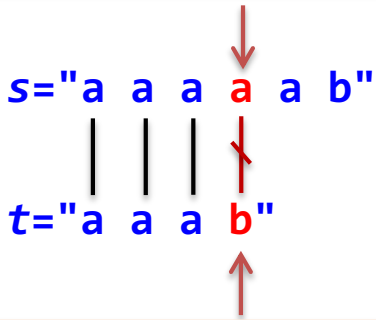
问题1

以目标串*s*="aaaaab"，模式串*t*="aaab"为例。

j	0	1	2	3
t[j]	a	a	a	b
next[j]	-1	0	1	2

j	0	1	2	3
t[j]	a	a	a	b
next[j]	-1	0	1	2

第1趟匹配



i=3

j=3

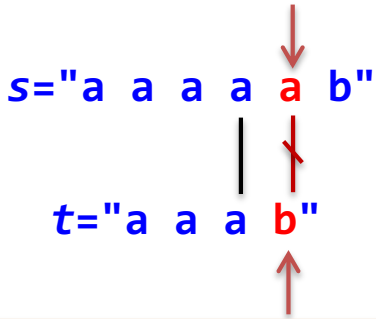
失败,修改为

i不变

j=next[3]=2

比较4次

第2趟匹配



i=4

j=3

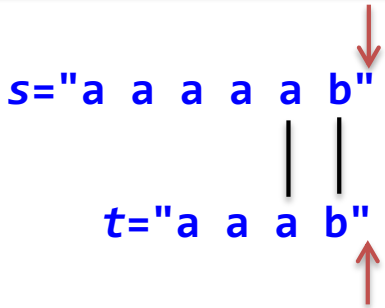
失败,修改为

i不变

j=next[3]=2

比较2次

第3趟匹配



i=6

j=4

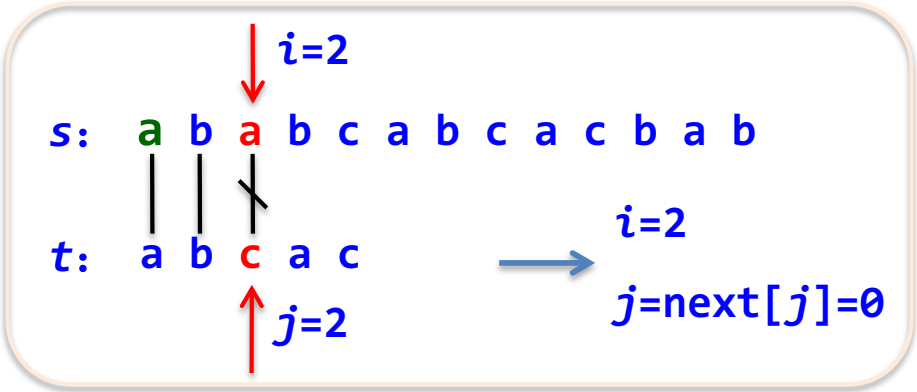
成功，返回2

比较2次

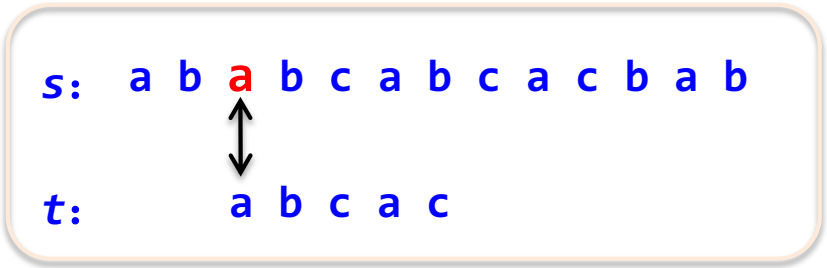
共比较8次<12次

问题2 以目标串s="ababcabcacbab"，模式串t="abcac"为例。

j	0	1	2	3	4
t[j]	a	b	c	a	c
next[j]	-1	0	0	0	1



跳过了s_1的那一趟？



- 失配处为s_2/t_2。有"s_1"="t_1"
- BF下一趟"$s_1s_2s_3\cdots$"/"$t_0t_1t_2\cdots$"
- next[2]=0 ⇨ "t_1"≠"t_0"
- 即"s_1"≠"t_0",所有s_1开始的匹配是没有必要的！

改进KMP算法

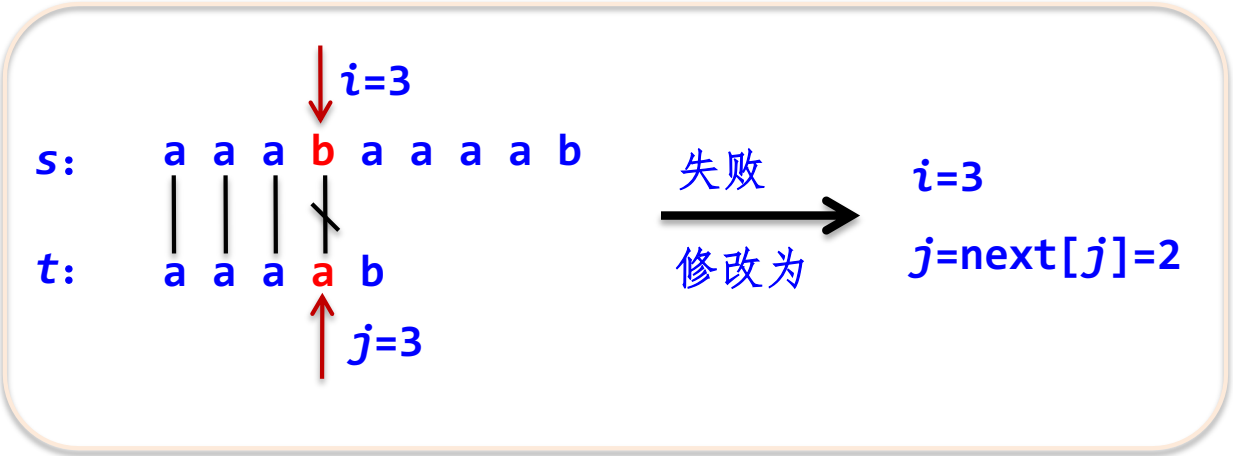
基本KMP算法存在的问题

设主串s="aaabaaaab"，模式串t="aaaab"。

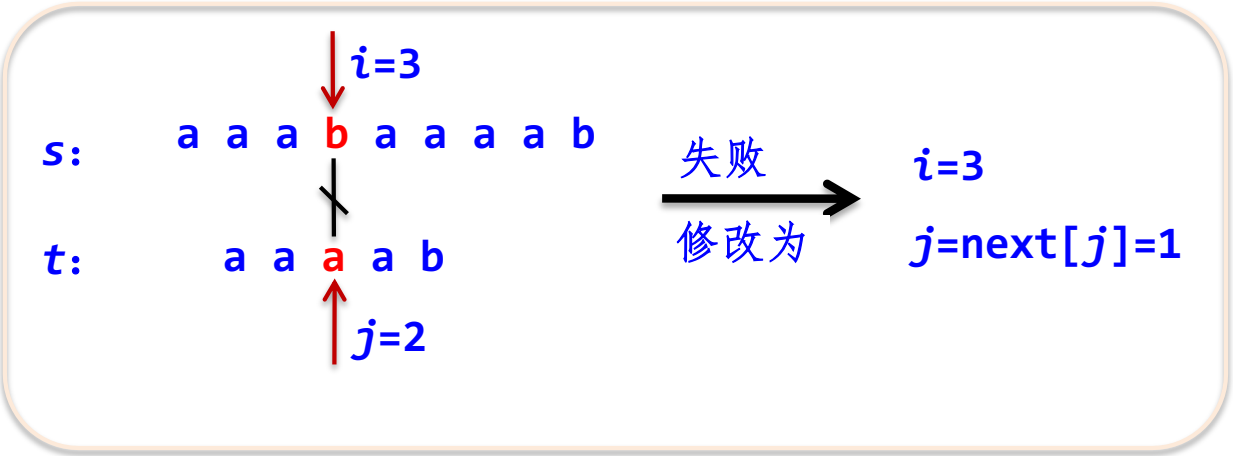
j	0	1	2	3	4
t[j]	a	a	a	a	b
next[j]	-1	0	1	2	3

j	0	1	2	3	4
t[j]	a	a	a	a	b
next[j]	-1	0	1	2	3

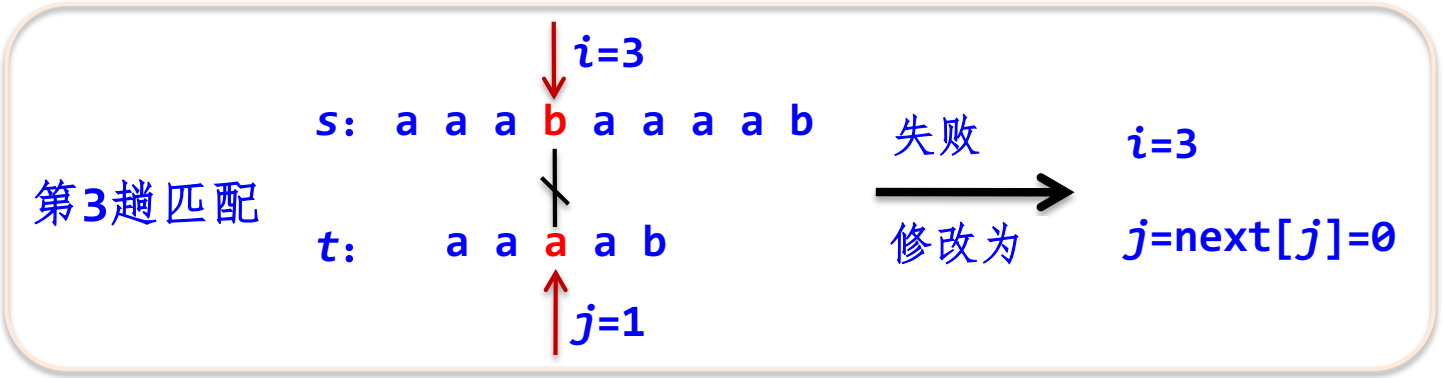
第1趟匹配

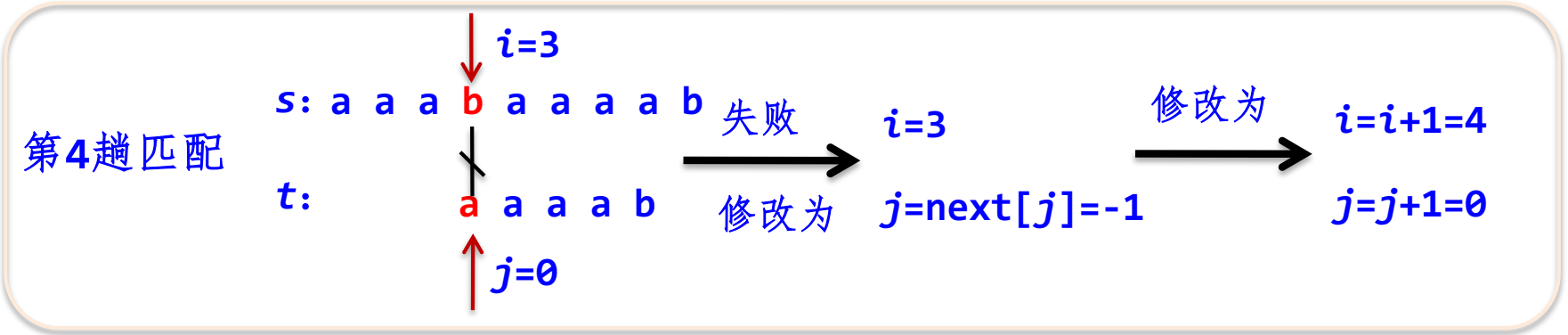


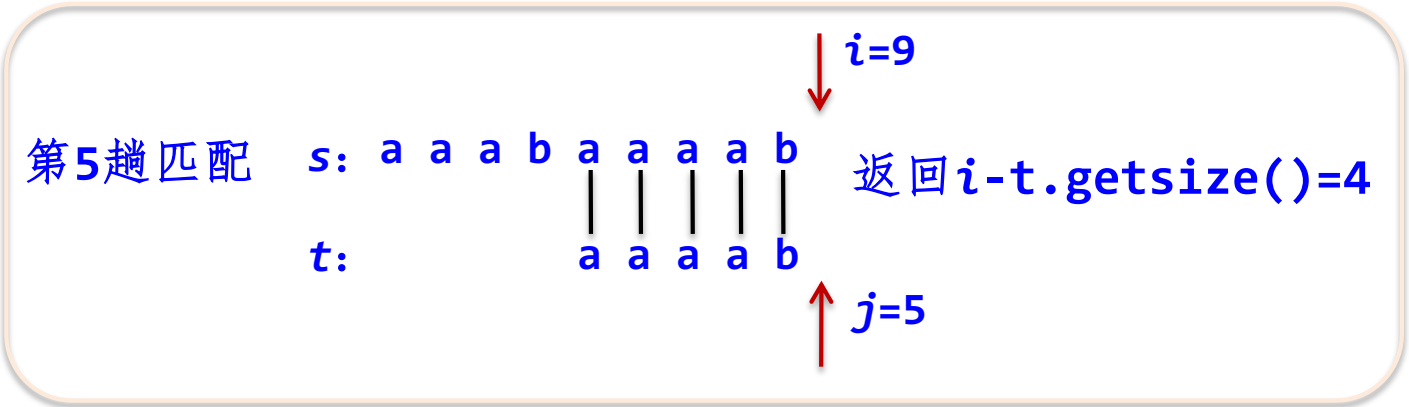
第2趟匹配



j	0	1	2	3	4
t[j]	a	a	a	a	b
next[j]	-1	0	1	2	3

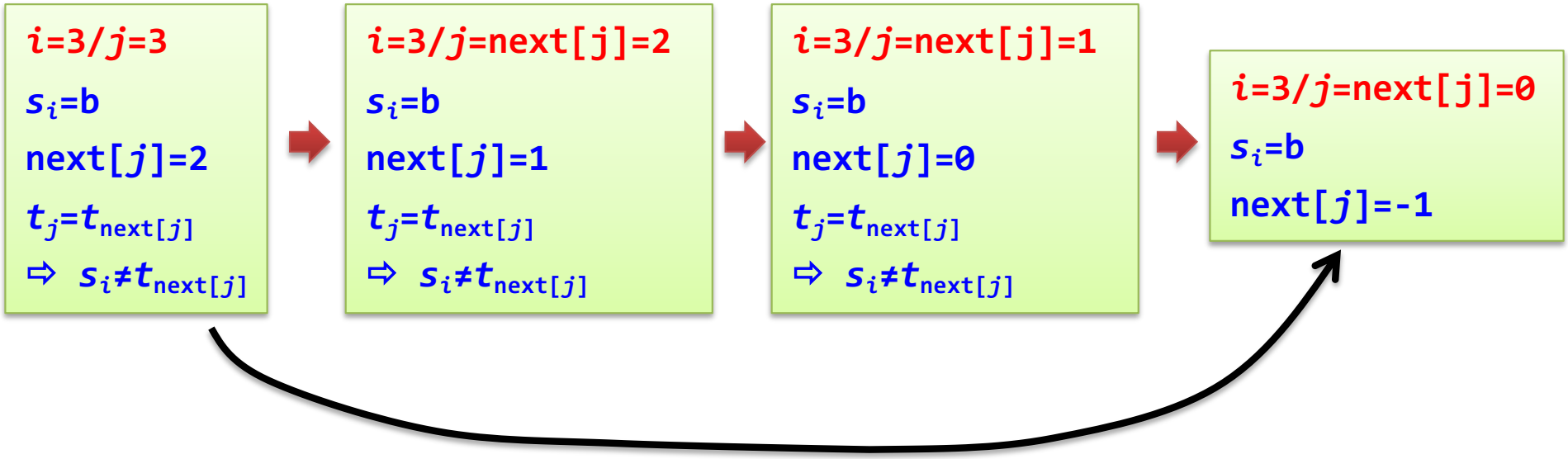






设主串s="aaabaaaab"，模式串t="aaaab"。

j	0	1	2	3	4
t[j]	a	a	a	a	b
next[j]	-1	0	1	2	3



j	0	1	2	3	4
$t[j]$	a	a	a	a	b
next[j]	-1	0	1	2	3
nextval[j]	-1	-1	-1	-1	3

- 首先，**nextval[0]=-1**
- **j=1**：失配处为s_i/t_1，则$s_i \neq t_1$。KMP算法的下一次比较$s_i/t_{next[1]}$，而**next[1]=0**，并且$t_0=t_1$，说明一定有$s_i \neq t_{next[1]}$ ⇨ **nextval[j]=nextval[next[j]]=-1**

j	0	1	2	3	4
$t[j]$	a	a	a	a	b
next[j]	-1	0	1	2	3
nextval[j]	-1	-1	-1	-1	3

将next数组改为nextval数组，与next[0]一样，先置nextval[0]=-1。假设求出next[j]=k，现在失配处为s_i/t_j，即$s_i \neq t_j$，

（1）如果有$t_j=t_k$成立，可以直接推出$s_i \neq t_k$成立，没有必要再做s_i/t_k的比较，直接置nextval[j]=nextval[k]（nextval[next[j]]），即下一步做$s_i/t_{\text{nextval}[j]}$的比较。

（2）如果有$t_j \neq t_k$，没有改进的，置nextval[j]=next[j]。

求出next：

- $t_j=t_k$：nextval[j]=nextval[k]
- 否则：nextval[j]=next[j]=k



```
void GetNextval(string t,int* nextval)          //由模式串t求出nextval值
{  int j=0,k=-1;
   nextval[0]=-1;
   while (j<t.length())
   {  if (k==-1 || t[j]==t[k])                  //k为-1或比较的字符相等时
      {  j++;k++;
         if (t[j]!=t[k])                        //两个字符不相等时
            nextval[j]=k;
         else
            nextval[j]=nextval[k];
      }
      else k=nextval[k];                        //k回退
   }
}
```

将next改为nextval即可

```
int KMPval(string s,string t)                    //改进的KMP算法
{  int n=s.length(),m=t.length();
   int* nextval=new int[m];
   GetNextval(t,nextval);                        //求出nextval数组
   int i=0,j=0;
   while (i<n && j<m)
   {  if (j==-1 || s[i]==t[j])
      {  i++;                                    //i,j各增1
         j++;
      }
      else j=nextval[j];                         //i不变,j回退
   }
   if (j>=m) return i-m;
   else return -1;
}
```

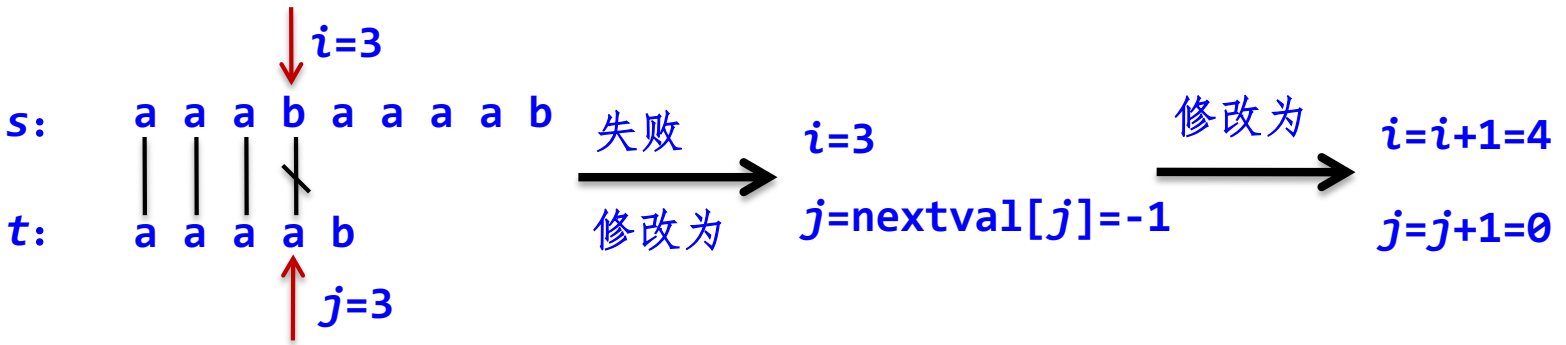
本算法的时间复杂度也为O(*n*+*m*)。

【例4.6】设s="aaabaaaab"，t="aaaab"。计算模式串t的nextval函数值。并画出利用改进KMP算法进行模式匹配时每一趟的匹配过程。

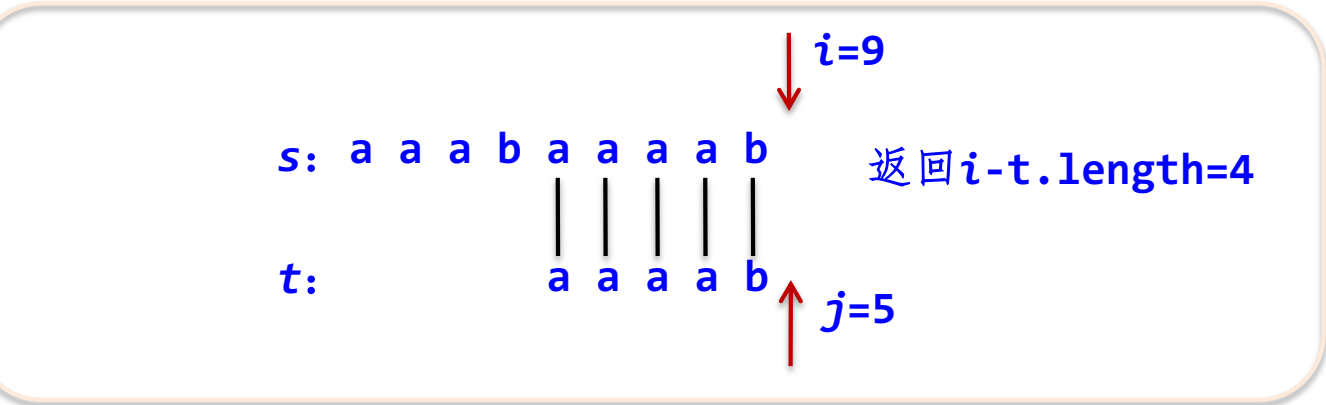
j	0	1	2	3	4
t[j]	a	a	a	a	b
next[j]	-1	0	1	2	3
nextval[j]	-1	-1	-1	-1	3

j	0	1	2	3	4
t[j]	a	a	a	a	b
next[j]	-1	0	1	2	3
nextval[j]	-1	-1	-1	-1	3

第1趟匹配



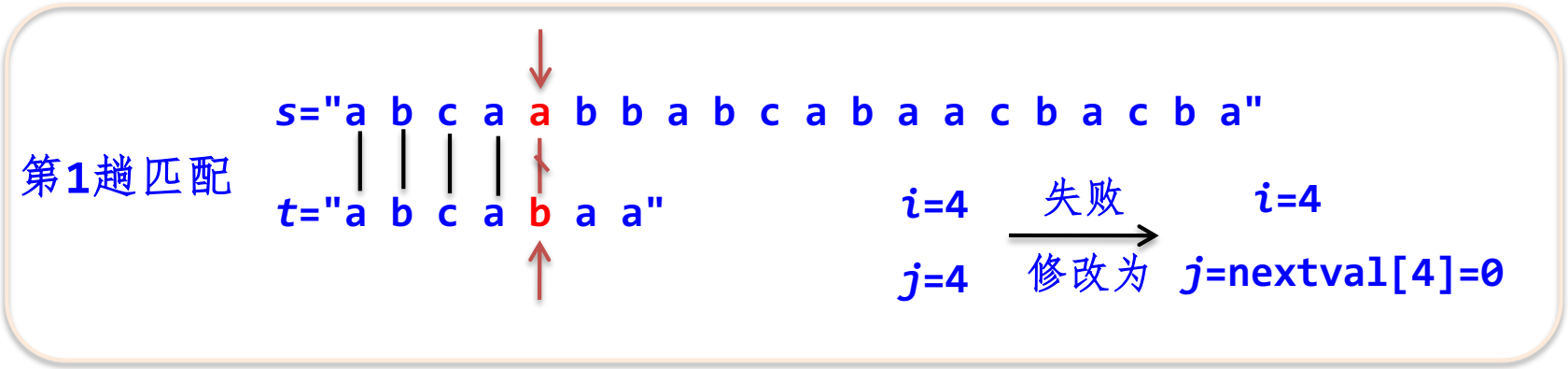
第2趟匹配

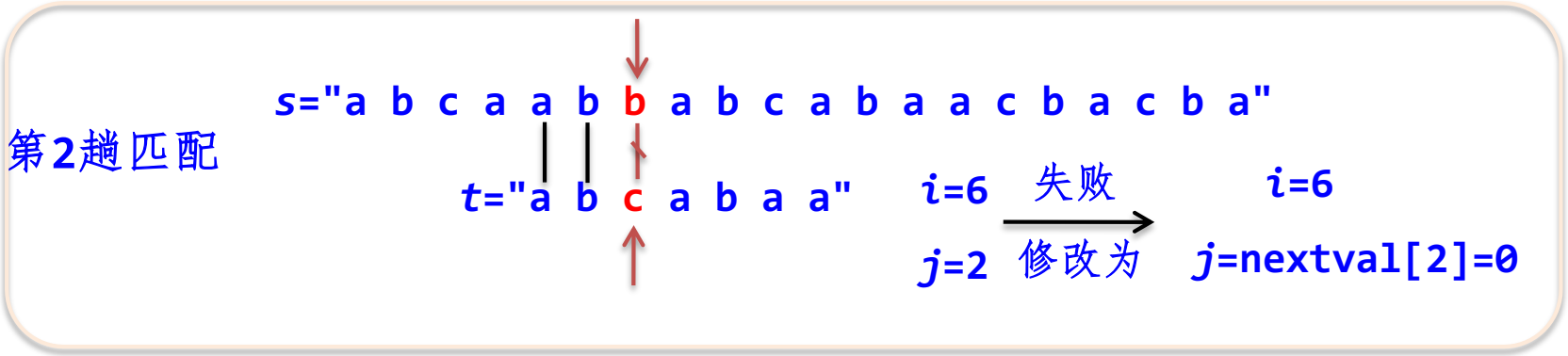


【例】设目标串为s="abcaabbabcabaacbacba"，模式串t="abcabaa"。计算模式串t的nextval函数值。并画出利用KMP算法进行模式匹配时每一趟的匹配过程。

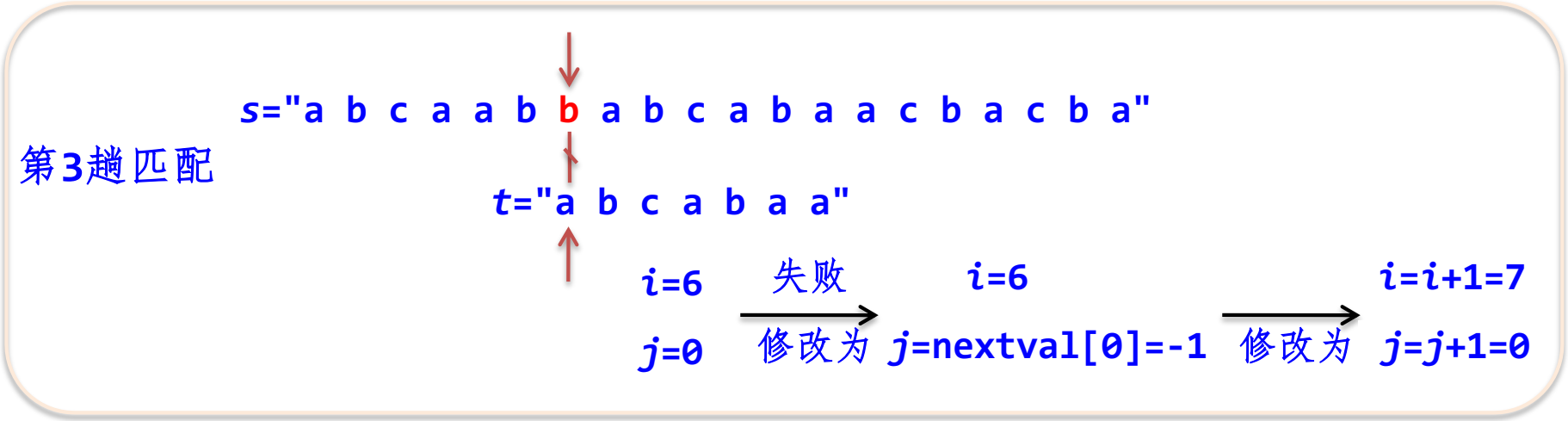
j	0	1	2	3	4	5	6
t[j]	a	b	c	a	b	a	a
next[j]	-1	0	0	0	1	2	1
nextval[j]	-1	0	0	-1	0	2	1

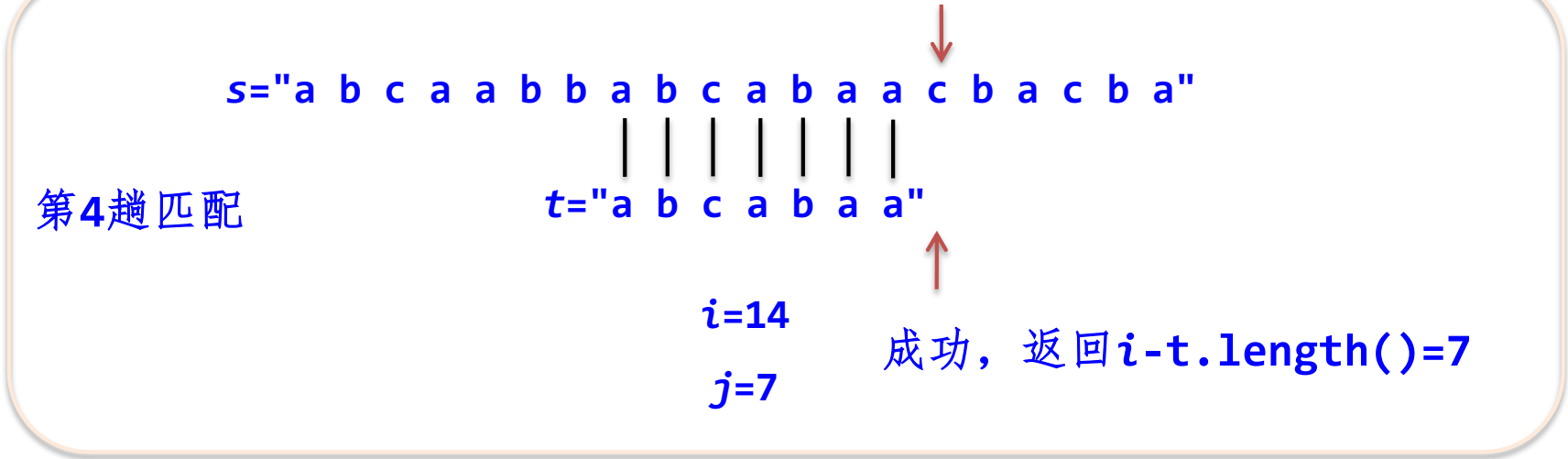
j	0	1	2	3	4	5	6
nextval[j]	-1	0	0	-1	0	2	1





j	0	1	2	3	4	5	6
nextval[j]	-1	0	0	-1	0	2	1





【课后练习】设s=”bababababcababcabab",
t=”ababcabab"。计算模式串t的nextval函数值。并画出利用改进KMP算法进行模式匹配时每一趟的匹配过程。

j	0	1	2	3	4	5	6	7	8
t[j]	a	b	a	b	c	a	b	a	b
next[j]									
nextval[j]									